

Judgment Geometry (JuGeo)

A Proof-Carrying Python Alternative to LEAN, Coq, and F*

Halley Young

2026

What This Talk Is About

*“By the end, you’ll understand a new approach to proving code correct —
one that works **with** Python, not against it.”*

- No category theory required
- No sheaf theory required
- Just Python and basic logic

The Stakes — Why Correctness Matters

Ariane 5 (1996)

- Integer overflow
- 64-bit → 16-bit cast
- **\$370M** lost in 37 s

Heartbleed (2014)

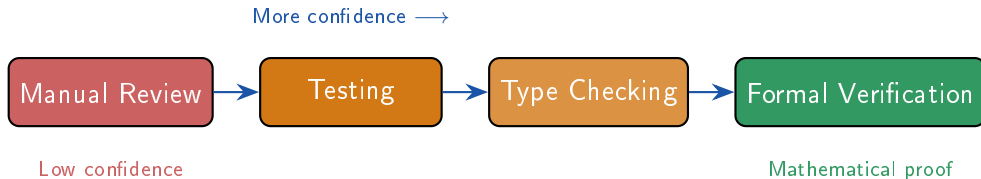
- Missing bounds check
- OpenSSL buffer over-read
- **500K+** servers exposed

Therac-25 (1985)

- Race condition
- Radiation therapy machine
- **6 patients** killed or injured

Bugs cost **lives** and **money**. Can we do better?

How Do We Know Code Is Correct?



- Each step catches *more* bugs — but demands *more* effort
- Most Python projects stop at testing
- **What would it take to go further?**

What Is Formal Verification?

Definition

A **mathematical proof** that code meets its specification — for *all* possible inputs.

What testing says

“It didn’t crash on **my** tests.”

What verification says

“It **cannot** have this class of bug.”

- Covers *infinite* input space
- Eliminates entire bug categories by construction
- The gold standard — but historically very expensive

The Big Three — LEAN, Coq, F*

- LEAN 4** Interactive theorem prover from [Microsoft Research](#).
Dependent types, tactic-based proofs, growing math library.
- Coq** Foundational proof assistant from [INRIA](#) (France).
Calculus of Inductive Constructions, used in CompCert & seL4.
- F*** Effectful verification language from [MSR](#).
Refinement types + SMT (Z3), used in Project Everest (TLS).

All three are **powerful**, **mature**, and **not Python**.

The Curry-Howard Idea

“Proofs are programs. Programs are proofs.”

Logic side

- Proposition P
- Proof of P
- $P \Rightarrow Q$

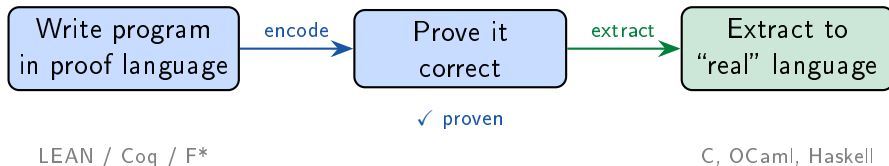
$\equiv$

Programming side

- Type P
- Value of type P
- Function $P \rightarrow Q$

- A proof of P is a **program** that has type P
- If the program type-checks, the proof is **valid**
- Types *enforce* correctness at compile time

The Extraction Model



- You re-implement your algorithm inside the proof assistant
- Prove properties about that implementation
- Extract a certified executable — in a language *they* choose
- **Your original code is never verified directly**

Pain Point 1 — No Python

Proof Assistant	Extraction Targets
LEAN 4	C
Coq	OCaml, Haskell
F*	OCaml, F#, C, Wasm
Python? Nowhere.	

- Python is the **#1 language** (TIOBE, Stack Overflow, GitHub)
- ML, data science, web backends, scripting — all Python
- Formal verification simply **ignores** the largest developer community

Pain Point 2 — Rewrite Everything

Your Python

```
def merge_sort(xs):  
    if len(xs) <= 1:  
        return xs  
    mid = len(xs) // 2  
    left = merge_sort(xs[:mid])  
    right = merge_sort(xs[mid:])  
    return merge(left, right)
```

Rewritten in Coq

```
Fixpoint merge_sort (xs: list nat)  
  : list nat :=  
  match xs with  
  | [] => []  
  | [x] => [x]  
  | _ => let (l,r) :=  
          split xs in  
          merge (merge_sort l)  
                (merge_sort r)  
  end.
```

- Two codebases to write, maintain, and keep in sync
- Your *actual* Python is never verified
- **Any divergence voids the guarantee**

Pain Point 3 — Binary Trust

Type-checks $\Rightarrow$ fully trusted

Doesn't type-check $\Rightarrow$ zero trust

- There's no middle ground
- What about mixed evidence?
 - “Z3 proved the core math”
 - “Tests covered the edge cases”
 - “AI suggested the invariant”
 - “Manual review checked the IO”
- **No way to combine or account for these**

Pain Point 4 — Opaque Failures

F* says

(error) Unknown assertion failed
Z3 timed out. No explanation.
No hint about *what* went wrong.

Coq says

Error: Tactic failure.
Which sub-goal? Why?
Good luck if you're not an expert.

- Failures are **unstructured** — a string, not data
- No machine-readable explanation of *what* failed
- No suggestion for *how* to fix it
- **Debugging proofs is harder than debugging code**

Pain Point 5 — Python Effects

`try / except`

`async / await`

`yield`

`with (context mgr)`

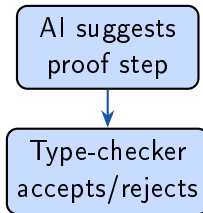
`metaclasses`

`descriptors`

- These are *core* Python idioms, not exotic features
- LEAN, Coq, F* have no native model for any of them
- F* has *some* effect support (monadic), but not Python's flavour
- **You can't verify what you can't model**

Pain Point 6 — AI Is External

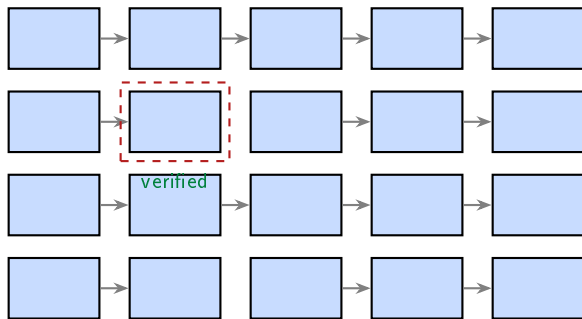
- GitHub Copilot for LEAN exists
- LLM-based tactic suggestion is growing
- But AI is a **bolt-on**:
 - No trust accounting
 - AI proof step = formal proof step
 - If the LLM hallucinates a lemma, the system accepts or rejects — nothing in between



Same trust as
human-written proof

- No way to say “AI contributed 40%, Z3 proved 60%”

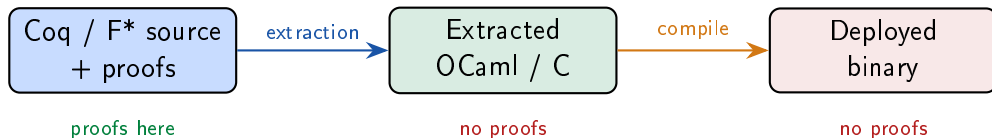
Pain Point 7 — Module-Level Only



Hundreds of files
One verified?

- Proof assistants verify *one module at a time*
- No built-in mechanism for cross-module reasoning
- Real projects have hundreds of interacting files
- **Verification doesn't scale to whole projects**

Pain Point 8 — Proofs Don't Ship



- Extracted code carries **no proof metadata**
- Deployed artifact has **no record** of how it was verified
- You must trust the build pipeline, not the artifact
- **Proofs evaporate at extraction time**

Summary — 8 Pain Points

#	Pain Point	Impact
1	No Python target	Largest community excluded
2	Must rewrite everything	Double codebase, drift risk
3	Binary trust (all or nothing)	Can't combine evidence sources
4	Opaque failures	Unstructured, unhelpful errors
5	Python effects unsupported	Core idioms can't be modeled
6	AI is a bolt-on	No trust accounting for LLMs
7	Module-level only	Doesn't scale to real projects
8	Proofs don't ship	Artifact carries no evidence

These aren't minor inconveniences —
they're **structural limitations** of the extraction model.

What If...?

What if we could...

- ✓ Verify **Python** directly — no rewriting
- ✓ Write proofs **in Python** — no new language
- ✓ Mix evidence sources with **trust accounting**
- ✓ Ship proofs **with** the code — not separate
- ✓ Scale verification to **entire projects**

“What if proofs traveled with the code?”

JuGeo

A **proof-carrying Python** framework
where proofs are **geometric objects**, not program terms.

Traditional

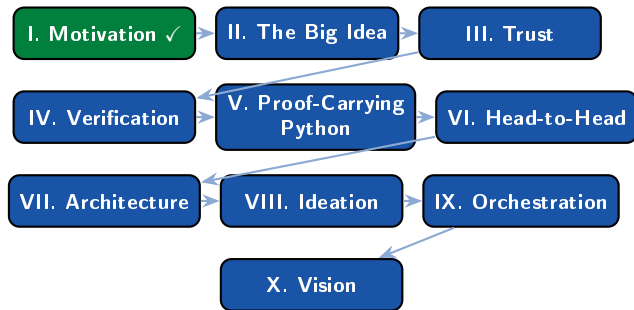
Proof = a program that type-checks

JuGeo

Proof = a geometric object attached to code

Proofs are **first-class data**, computed, composed, and shipped alongside your Python.

Roadmap



Part I complete — next: **The Big Idea** behind Judgment Geometry

Part II

The Big Idea: Geometry, Not Type Theory

Replace proof terms with landscapes of evidence

Two Ways to Think About Proofs

Traditional (Curry-Howard)

A proof is a **PROGRAM**

- A term in a typed lambda calculus
- A syntax tree
- Grammar: right or wrong

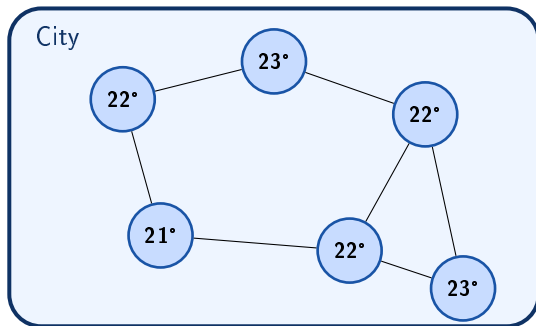
JuGeo Approach

A proof is a **MAP**

- A landscape of evidence
- Local measurements, globally consistent
- Geography: explore and chart

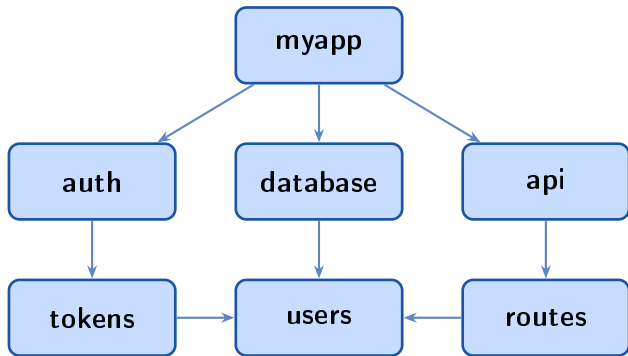
Think: checking grammar vs. charting a landscape

Analogy: Weather Forecasting



- Each station measures **local** temperature
- Neighbors **agree** $\Rightarrow$ glue into a global temperature map
- This idea is the heart of JuGeo's approach to proofs

Your Codebase Is a Map



Each function, module, or class is a **location** on the map.

JuGeo calls these locations **coordinates**.

What Is a Coordinate?

Definition

A **coordinate** is a specific place in your codebase:

```
module . function . line
```

Example

```
myapp.auth.validate_token
```

is a coordinate pointing to the `validate_token` function inside the `auth` module of `myapp`.

Like GPS coordinates — but for code

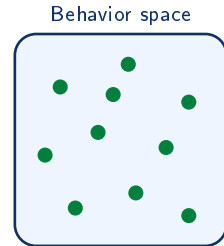
What Are Covers?

Definition

A **cover** is a collection of test inputs that together cover the behavior space.

Example

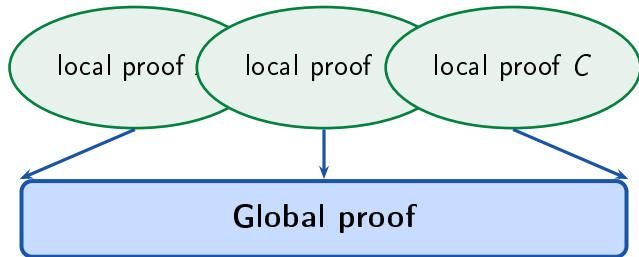
10 test inputs that exercise **all code paths** of a function.



Enough stations = full coverage

What Is Descent?

Descent = gluing local truths into a global truth



- Every test input confirms the spec ✓
- Test inputs cover everything ✓

⇒ Spec holds **globally**

When Gluing Fails — Obstructions



Obstruction

Neighboring proofs **contradict** each other at overlaps.

- Traditional tools: “Unknown” or “timeout” — no explanation
- JuGeo: a structured **mathematical object** that tells you **what** broke and **how** to fix it

The Core Insight

Traditional

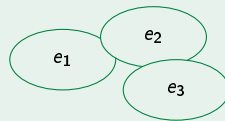
Proof = one big term

type-checks?

Yes or No

JuGeo

Proof = local evidence



Glued by **descent**

Structured failures when gluing
breaks → **actionable diagnostics**

The Judgment — JuGeo's Central Object

A **judgment** records:

WHAT is claimed

WHERE it lives

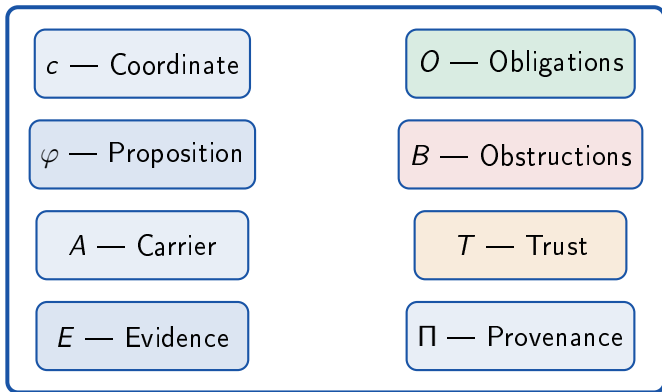
WHAT evidence supports it

HOW MUCH you should trust it

- Not a boolean (True/False)
- Not a proof term (a lambda expression)
- An **8-component record** — the central data structure of JuGeo

The Judgment 8-Tuple

$$\mathcal{J} = (c, \varphi, A, E, O, B, T, \Pi)$$



We'll walk through each component next.

Component 1: Coordinate (c)

in the codebase

A coordinate pinpoints the exact location being verified.

Example

```
myapp.auth.validate_token.line_42
```



Like **GPS coordinates** — but for code.

Component 2: Proposition (φ)

is claimed

The logical statement being verified at coordinate c .

Example

“validate_token returns a non-negative integer”

$$\varphi \equiv \forall x. \text{validate_token}(x) \geq 0$$

c tells you **where** to look; φ tells you **what** to check.

Component 3: Carrier (A)

of the claim

The mathematical domain in which φ is stated.

Example

$$A = \text{int} \rightarrow \text{int}$$

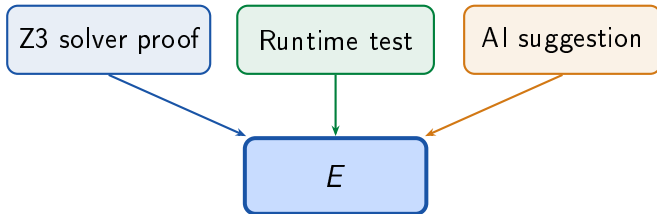
The function maps integers to integers.

- JuGeo's version of *dependent types*
- Determines which proof methods are valid
- Enables the system to compose proofs across modules

Component 4: Evidence Bundle (E)

— from multiple channels

Not one proof, but a **bundle** of evidence.



Like having **multiple witnesses** in court—
each adds confidence from a different angle.

Component 5: Obligations (O)

— what still needs proving

Obligations are proof goals that remain open.

✓ Arithmetic bounds — *discharged by Z3*

✓ Edge case $x = 0$ — *discharged by test*

? Concurrency safety — *open*

- Some discharged by **Z3**, some by **tests**
- Open obligations are tracked, not forgotten
- JuGeo never silently drops proof goals

Component 6: Obstructions (B)

Persistent

When something **can't** be proved, the failure itself becomes data.

F* / Lean

Unknown
(timeout after 30s)
No explanation.

JuGeo

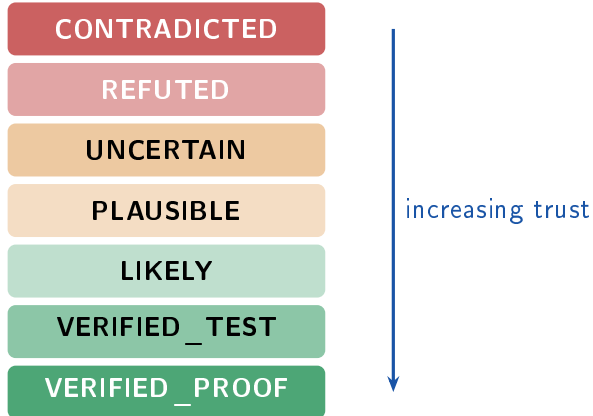
Obstruction:
overlap at auth $\cap$ db
type mismatch int/str
What broke + **how** to fix it.

Obstructions are **first-class mathematical objects**,
not error messages.

Component 7: Trust ($\mathcal{T}$)

to trust this judgment

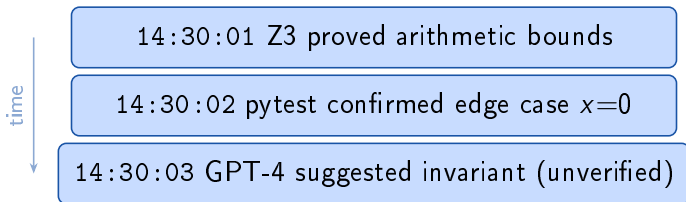
Not binary! A 7-level algebra.



Component 8: Provenance (Π)

produced this judgment and

Full audit trail for every piece of evidence.



- Every evidence channel is **timestamped** and **attributed**
- Enables reproducibility and blame tracking

Putting It All Together

A judgment for `validate_token`:

c

`myapp.auth.validate_token`

φ

$\forall x. \text{validate_token}(x) \geq 0$

A

`str` $\rightarrow$ `int`

E

Z3 proof + 42 passing tests + GPT-4 invariant

O

concurrency safety (open)

B

$\emptyset$ — no obstructions

T

VERIFIED_TEST (level 6 of 7)

$\square$

Z3 14:30 pytest 14:31 GPT-4 14:32

Part III

Trust — The Key Innovation

Not all evidence is created equal

The Problem with Binary Trust

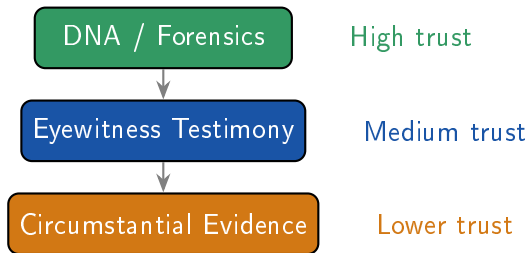
✓ Type-checks $\Rightarrow$ **100% trusted**

✗ Doesn't $\Rightarrow$ **0% trusted**

??? *Nothing in between*

- LEAN / Coq / F*: proof accepted or rejected — that's it
- Real-world evidence has **degrees of confidence**
- Z3 proved 80%, tests covered 15%, expert reviewed 5% — where does that go?

Analogy — Courtroom Evidence



- A jury doesn't rely on a single witness
- They **weigh** multiple channels of evidence
- The verdict reflects the *combined* picture
- **JuGeo works the same way for code verification**

JuGeo's Evidence Channels

Evidence Sources

1. Formal Proof (kernel)

2. SMT Solver (Z3)

3. Runtime Tests

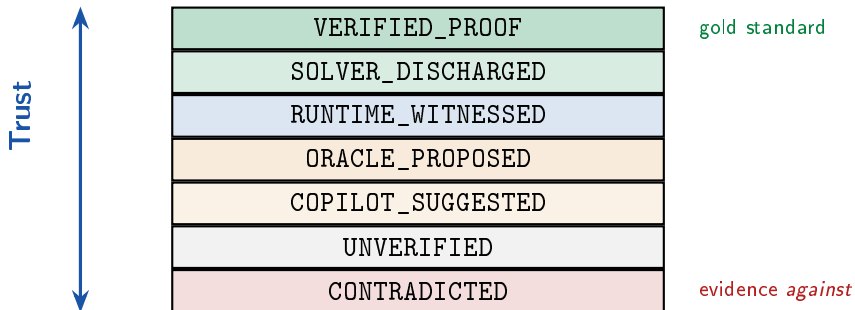
4. AI / Copilot Suggestions

5. Human Oracle Review

6. Composed Evidence

- Each channel produces evidence with a **different trust level**

The Seven Trust Levels



Every claim in JuGeo carries exactly one of these levels.

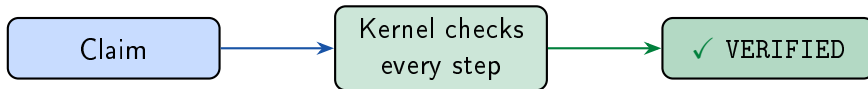
Higher is stronger; lower is weaker.

Trust Level — VERIFIED_PROOF

The Gold Standard

A **complete formal proof** checked by JuGeo's kernel.

Like a mathematical theorem — **certain** (relative to the kernel's axioms).



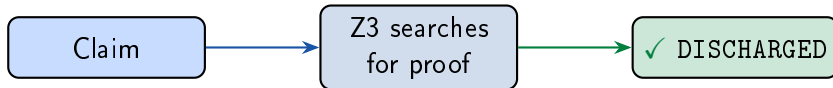
- No external dependency — pure logical deduction
- Analogous to the verdict of a flawless DNA match
- **Highest trust level in the system**

Trust Level — SOLVER_DISCHARGED

SMT Solver Proved It

Z3 (or another SMT solver) automatically proved the claim.

Very high confidence — but depends on Z3's own correctness.

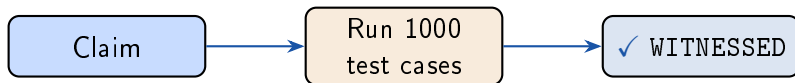


- Z3 is powerful but not infallible (solver bugs, bit-blasting limits)
- Excellent for arithmetic, array bounds, linear constraints
- **One step below a full kernel proof**

Trust Level — RUNTIME_WITNESSED

Tests Ran and Passed

Dynamic evidence: the claim held for *all tested inputs*.
Good evidence, but **only covers the tested cases**.



- “I checked 1 000 cases and they all worked”
- Does **not** guarantee the 1001st case
- Useful when solvers time out or formal proof is too expensive
- **Honest about its limits — unlike “all tests pass = correct”**

Trust Level — COPILOT_SUGGESTED

An AI Suggested This Is Correct

A language model (e.g. GitHub Copilot) proposed the claim or invariant.
Useful hint, but **NOT** a proof.



- AI contributions are **tracked, not banned**
- Cannot be silently promoted to VERIFIED_PROOF
- Must be independently confirmed to raise trust level

Trust Levels — The Lower Tiers

ORACLE_PROPOSED

- A human expert asserted this
- Reviewed but not mechanically checked
- “Trust me, I’m a domain expert”

UNVERIFIED

- No evidence yet
- Default state for any new claim
- “We haven’t looked at this”

CONTRADICTED

- Evidence **against** the claim
- A test failed, a counterexample found
- **Cannot be ignored**

CONTRADICTED is special: it **poisons** any composition it enters.
You cannot hide a known failure.

The Trust Algebra — Formal Definition

$$\mathcal{T} = (E_{\text{adm}}, \leq, \oplus, \ominus, \uparrow, \downarrow)$$

Don't panic! We'll explain each piece.

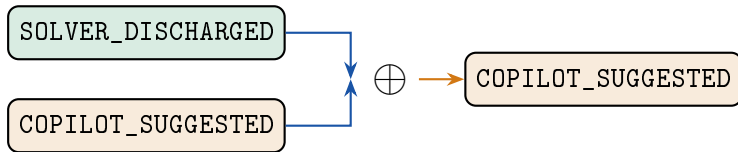
Symbol	Meaning
E_{adm}	The seven admissible trust levels
$\leq$	Ordering: CONTRADICTED < ... < VERIFIED_PROOF
$\oplus$	Compose evidence (take the weaker)
$\ominus$	Challenge evidence (may demote)
$\uparrow$	Promote (explicit, audited, never silent)
$\downarrow$	Demote (when evidence is weakened)

Six operators, three laws, one guarantee: **honest accounting**.

Operator — Composition ($\oplus$)

Rule: Take the Weaker Level

When combining evidence from two channels, the result is the **minimum**.



"A chain is only as strong as its weakest link."

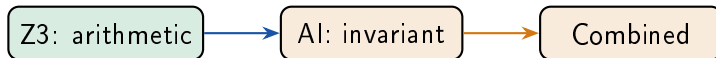
Why Weakest-Link?

Scenario

Z3 proved the **arithmetic** is correct.

A Copilot suggested the **loop invariant**.

Your overall confidence is limited by the Copilot suggestion.

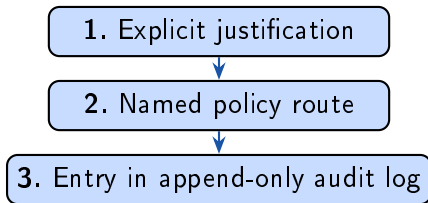


- **You cannot launder low-trust evidence through high-trust channels**
- The weakest link determines overall strength
- This is by design — it keeps the system **honest**

Law 1 — No Silent Promotion

The Rule

You **CANNOT** silently upgrade trust. Every promotion requires:



- Attempting silent promotion $\Rightarrow$ **JuGeoError**
- The audit log is **append-only**: no rewriting history
- **Trust must be earned, not assigned**

Silent Promotion — Fails

```
from jugeo.trust import TrustProfile, COPILOT_SUGGESTED, VERIFIED_PROOF

profile = TrustProfile(
    tier=COPILOT_SUGGESTED,
    source="copilot-invariant-suggestion",
    claim="loop_invariant_holds",
)

# Attempt silent promotion
profile.promote(VERIFIED_PROOF)
# → raises JuGeoError:
# "Silent promotion from COPILOT_SUGGESTED
# to VERIFIED_PROOF is forbidden.
# Provide explicit=True with a reason."
```

The system **refuses** to upgrade trust without justification.
No shortcuts.

Explicit Promotion — Works

```
from jugeo.trust import TrustProfile
from jugeo.trust import COPILOT_SUGGESTED, SOLVER_DISCHARGED

profile = TrustProfile(
    tier=COPILOT_SUGGESTED,
    source="copilot-invariant-suggestion",
)

# Explicit promotion with full justification
promoted = profile.promote(
    SOLVER_DISCHARGED,
    explicit=True,
    reason="z3-confirmed-arithmetic",
)

# ✓ promoted.tier == SOLVER_DISCHARGED
# ✓ Audit trail records promotion + reason
```

Every promotion is **justified, named, and logged**.

Law 2 — Conservative Join

Rule: Always Take the Weaker Trust Level

$a \oplus b = \min(a, b)$. This prevents **trust laundering**.

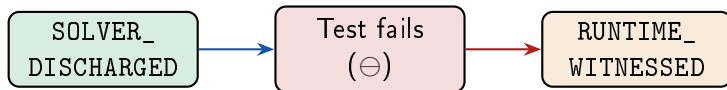
Channel A	$\oplus$	Channel B	=	Result
VERIFIED_PROOF	$\oplus$	VERIFIED_PROOF	=	VERIFIED_PROOF
SOLVER_DISCHARGED	$\oplus$	RUNTIME_WITNESSED	=	RUNTIME_WITNESSED
SOLVER_DISCHARGED	$\oplus$	COPILOT_SUGGESTED	=	COPILOT_SUGGESTED
VERIFIED_PROOF	$\oplus$	CONTRADICTED	=	CONTRADICTED

- CONTRADICTED poisons everything — you cannot ignore failure
- Composition is commutative: $a \oplus b = b \oplus a$
- **No amount of strong evidence cancels a known contradiction**

Law 3 — Challenge Conservativity

Rule: Challenges May Demote, Never Ignore

When evidence is challenged (e.g. a test fails), the system **must** respond. Old trust cannot stand without explanation.



- Like recalling a witness's testimony in court
- The demotion and its reason are logged in the audit trail
- **The system never pretends a challenge didn't happen**

A Real Scenario

Verifying `binary_search`

Cover points: 10 total obligations

Z3 proved: 8 of 10

SOLVER_DISCHARGED

Z3 timed out: 2 edge cases

--

Runtime tests: confirmed the 2 edge cases

RUNTIME_WITNESSED

JuGeo's Honest Report

- Overall trust: RUNTIME_WITNESSED (conservative join)
- Residual obligations: **none**
- Every channel's contribution: **recorded**

Trust Models — Comparison

	LEAN	Coq	F*	JuGeo
Trust levels	2 (binary)	2 (binary)	2 (binary)	7
Evidence channels	1	1	2 (SMT)	6
Audit trail	No	No	No	Yes
AI accounting	No	No	No	Yes
Challenge protocol	No	No	No	Yes
Promotion control	N/A	N/A	N/A	Yes

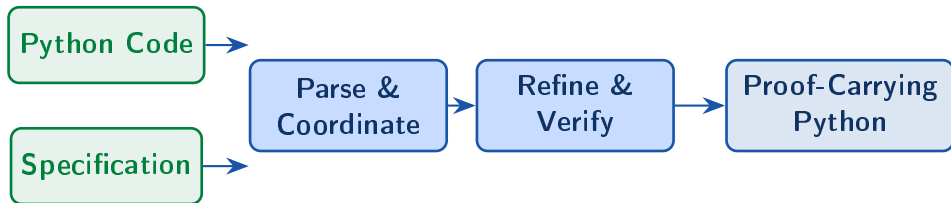
JuGeo is the only system that gives you an **honest, multi-channel account of verification evidence.**

Part IV

How Verification Works in JuGeo

Refinement types, effects, and the verification pipeline

Overview: The Verification Pipeline



- **Input:** your Python code + a specification of desired behavior
- **Pipeline:** parse, assign coordinates, refine types, verify
- **Output:** original Python with machine-checked proofs attached

What Are Refinement Types?

Idea

A **refinement type** = a base type + a logical condition.

$$\{x : \text{int} \mid x \geq 0\}$$

int alone

x could be -1, 0, 42, ...

{x:int | x>=0}

Only 0, 1, 2, ... allowed

More precise than plain types — catches more bugs at compile time.

Refinement Types in F*

F* syntax

```
val abs : x:int -> Tot (y:int{y >= 0})  
let abs x = if x >= 0 then x else -x
```

- F* pioneered refinement types with **SMT (Z3)** discharge
- Write the refinement $\rightarrow$ Z3 checks it automatically
- Powerful — but two catches:

Catch 1

You must write in **F*'s language**, not Python

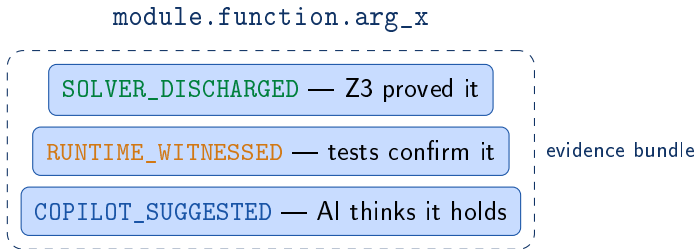
Catch 2

Trust is **binary**: proved or not — no partial evidence

Refinement Types in JuGeo

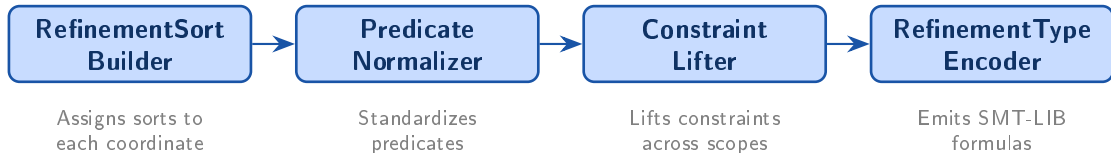
Key Difference

Every refinement lives at a **coordinate** with its own **evidence bundle**.



Three sources, tracked separately — combine or compare.

The Refinement Pipeline



- 1 **Sort**: determine the logical sort of each refinement
- 2 **Normalize**: rewrite predicates into canonical form
- 3 **Lift**: propagate constraints from callees to callers
- 4 **Encode**: translate into SMT-ready assertions for Z3

What Are Effects?

Definition

An **effect** is anything a function does **besides** returning a value.

Exceptions

Mutation

I/O

Async

Generators

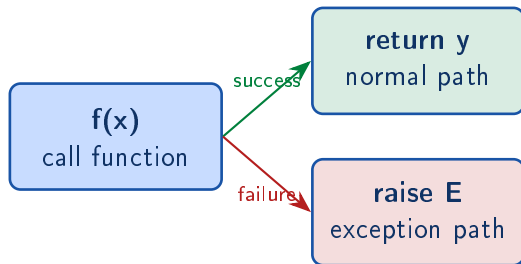
Context Mgrs

Metaclasses

Python is **effect-rich** — almost every function has side effects.

Verification must account for **all** of them.

Why Effects Matter for Verification



- A function that can throw must be verified along **both** paths
- Same for async (suspend/resume), generators (yield/send), context managers (`__enter__`/`__exit__`)
- Ignoring effects = ignoring half the behavior

Effects in F*: Dijkstra Monads

F*'s Approach

Model each effect as a **Dijkstra monad** — a mathematical object encoding the effect's weakest-precondition transformer.

Example: F* stateful effect

```
effect ST (a:Type) (pre: heap -> Type)
  (post: heap -> a -> heap -> Type) =
  STATE a (fun p h -> pre h /\ forall x h'. post h x h' ==> p x h')
```

Strengths

- Mathematically precise
- Compositional

Limitations

- Very abstract notation
- No native Python effects

Effects in JuGeo: Geometric, Not Monadic

Monadic (F^*)

- Effects wrapped in monads
- Compose via **monad transformers**
- Encoding overhead

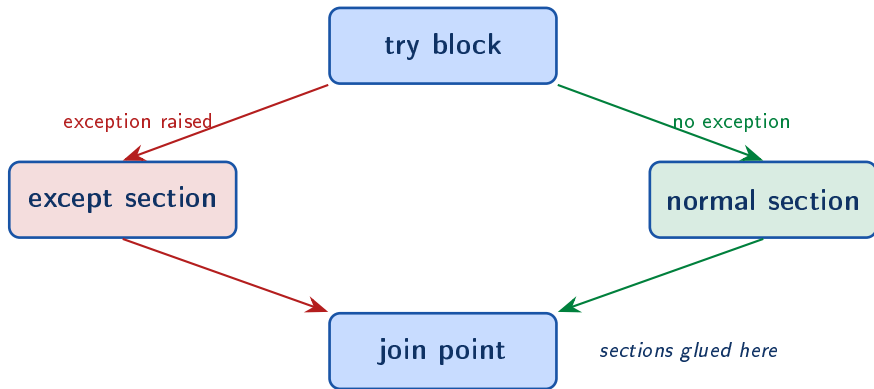
Geometric (JuGeo)

- Effects are **sections** in the site
- Compose via **gluing**
- Native Python semantics

*Key insight: effects compose via
gluing, not monad transformers.*

Each effect is a geometric structure on the coordinate space.

Python Exceptions as Coordinate Forks



try/except creates a **fork** in coordinate space — two branches that must be **glued** at the join point.

Exception Example

Python code

```
try:  
    result = 10 / x  
except ZeroDivisionError:  
    result = 0
```

JuGeo model

$x \neq 0$
 $\text{result} = 10/x$

$x = 0$
 $\text{result} = 0$

Both branches verified

- **Branch 1:** $x \neq 0$ — normal division, $\text{result} = 10/x$
- **Branch 2:** $x = 0$ — exception caught, $\text{result} = 0$
- Both sections glued $\Rightarrow$ result always has a valid value

Python Async as Suspended Morphisms



Coroutine A **suspended section morphism** — paused mid-execution

await A resume point where control may transfer

Scheduling Morphism composition across tasks

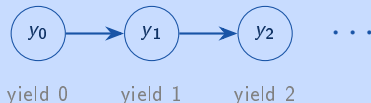
Cancellation An **obstruction** — the section cannot be completed

Python Generators as Fiber Sequences

Generator code

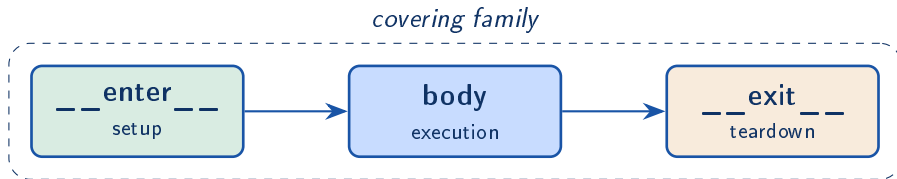
```
def counter(n):  
    i = 0  
    while i < n:  
        yield i  
        i += 1
```

JuGeo model



- `yield` emits a **partial section** — one element of the fiber
- `send()` injects a value into the generator fiber
- `close()` terminates the section
- Each `yield` point is a **coordinate** in the sequence

Python Context Managers as Covers



- A with-block is a **covering family** with three phases
- `--enter--`: acquire the resource (setup morphism)
- `body`: use the resource (section)
- `--exit--`: release the resource (teardown morphism)
- The cover **guarantees** teardown runs — even on exception

Context Manager Example

Python code

```
with open("file.txt") as f:  
    data = f.read()  
    # f is closed here
```

JuGeo model

enter: `f = open(...)`

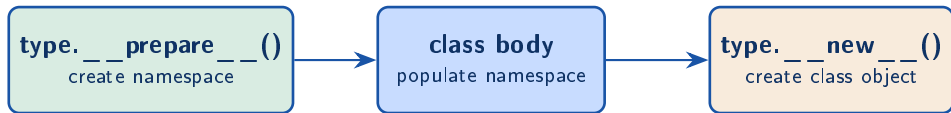
body: `data = f.read()`

exit: `f.close()`

Temporal obligation

JuGeo encodes: `__exit__` **MUST** execute after the body completes — whether normally or via exception. This is a provable property, not just convention.

Python Metaclasses



- Metaclass creation is a **three-phase morphism sequence**
- Each phase transforms the namespace at a distinct coordinate
- JuGeo verifies the contract across all three phases

No other proof assistant models Python metaclass creation

Effect Comparison: JuGeo vs. the Big Three

Effect	F*	LEAN	Coq	JuGeo
Exceptions	custom	custom	—	native
State mutation	Dijkstra	custom	—	native
Async / await	—	—	—	native
Generators	—	—	—	native
Context managers	—	—	—	native
Metaclasses	—	—	—	native

— = no support **custom** = requires manual encoding
native = built into the verification framework

Why Geometric Effects Are Better

① Visible overlap conditions

- Effects are explicit in the coordinate structure
- Not hidden inside monads — you can see where effects interact

② Native Python effects

- No encoding step — `try/except`, `async/await`, `yield`, with all modeled directly
- Python semantics in, Python semantics out

③ Persistent obstructions

- When an effect *fails* to compose, the obstruction is recorded
- Like a “proof debt” — visible, trackable, resolvable

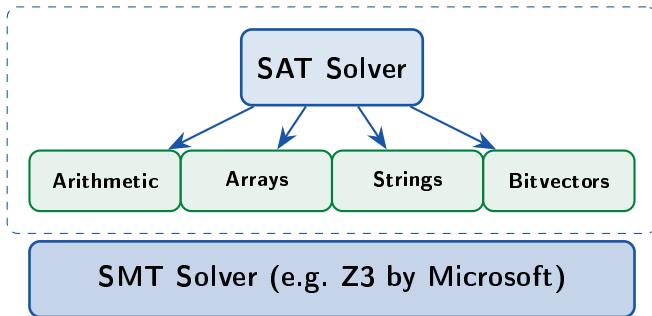
JuGeo is the **first proof assistant** that natively models
all of Python's effect system:

exceptions async/await generators
context managers metaclasses

— as **geometric structures** on the semantic site.

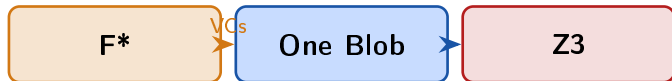
Next: how JuGeo's coordinate system enables incremental verification.

What Is an SMT Solver?



- **SAT** + **theories** = SMT
- Automatically **proves or disproves** logical formulas
- Returns sat, unsat, or unknown

How F* Uses Z3

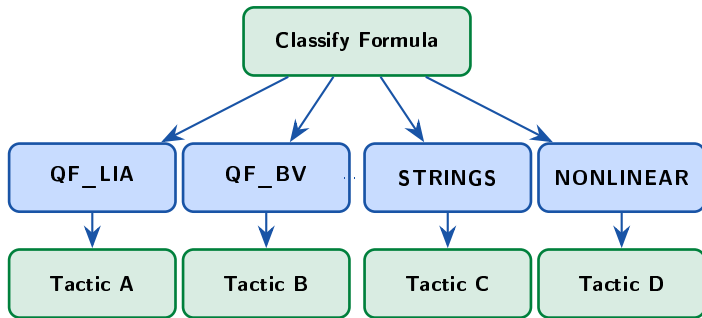


sat / unsat / unknown

The Problem

- Sends **all** verification conditions as a single query
- **No fragment awareness** — treats linear arithmetic the same as strings
- **No fallback strategy** — if Z3 says **unknown**, you're stuck
- **No structured result** — just a boolean answer

JuGeo's Fragment-Aware Approach



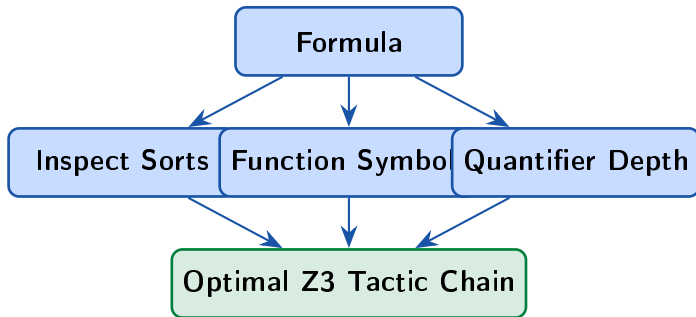
Key Innovation

Classify the formula **before** sending to Z3.
Different math $\Rightarrow$ different solving strategies.
JuGeo recognizes **13 decidable fragments**.

The 13 SMT Fragments

Fragment	Domain	Timeout
QF_LIA	Linear integer arithmetic	5 s
QF_LRA	Linear real arithmetic	5 s
QF_BV	Bitvectors	3 s
QF_UF	Uninterpreted functions	2 s
QF_AUFLIA	Arrays + UF + LIA	8 s
QF_ABV	Arrays + bitvectors	6 s
STRINGS	String constraints	4 s
SEQUENCES	Sequence reasoning	6 s
ARRAYS	Pure array theory	4 s
DATATYPES	Algebraic datatypes	5 s
NONLINEAR	Nonlinear arithmetic	15 s
QUANTIFIED	Universally quantified	20 s
MIXED	Multi-theory combination	10 s

Fragment Classification



FragmentClassifier

Like a **doctor diagnosing before prescribing treatment**:

Examines sorts, symbols, and quantifiers → routes to the best strategy.

Five Routing Strategies

Cheapest — minimize cost

Fastest — minimize latency

MostTrusted — maximize trust

RoundRobin — load balance

Smart — adaptive (recommended)

Choose based on your project's priorities:

production → **MostTrusted** CI/CD → **Fastest** exploration → **Smart**

Jurisdiction Management



Escalation Chain

- 1 Z3 handles decidable fragments → **highest trust**
- 2 Outside Z3's jurisdiction? Try **runtime witnesses**
- 3 Still stuck? Defer to **copilot oracle** with lower trust ceiling

Auditable Proofs from Z3

F* Gets...

- sat
- unsat
- unknown

That's it.

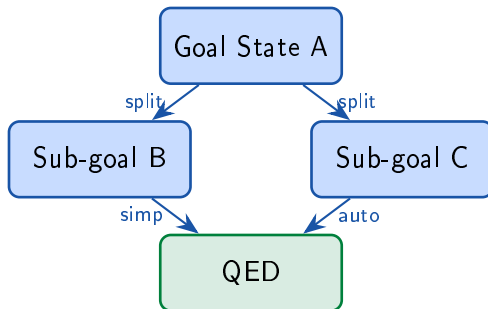
JuGeo Gets...

- Proof object
- Unsat-core extraction
- Trust level
- Latency metrics

Structured Z3Result

```
class Z3Result:
    verdict: Literal["sat","unsat","unknown"]
    proof_object: Optional[Z3Proof]
    unsat_core: list[str]
    trust_level: float      # 0.0 .. 1.0
    latency_ms: float
```


What Are Tactics?



Definition

Tactics are commands that manipulate proof state:

- “Try this strategy,” “split into cases,” “apply this lemma”
- They are proof **scripts** — sequences of instructions

Tactics in LEAN / Coq / F*

System	Mechanism	Example Tactics
LEAN	tactic mode	unfold, induction, simp, omega
Coq	Ltac / Ltac2	intro, apply, rewrite, auto
F*	Meta-F* Tac	norm, smt, trefl, dump

Limitation: They Only Touch Proof Terms

- Cannot manipulate **trust levels**
- Cannot manipulate **evidence quality**
- Cannot trigger **repair** when proofs break
- No notion of **geometric structure**

JuGeo's Semantic Moves

Replace Tactics with Semantic Moves

First-class proof operations on the **full judgment geometry**.

```
@dataclass
class SemanticMove:
    id: str
    kind: MoveKind
    target: Coordinate
    preconditions: list[Predicate]
    postconditions: list[Predicate]
    expected_gain: float
    cost_estimate: float
    trust_floor: float      # minimum trust required
```

Moves operate on **geometry + trust + evidence**,
not just proof terms.

Move Vocabulary — Structural Rules

WEAKEN

Add an unused hypothesis

CONTRACT

Merge duplicate hypotheses

EXCHANGE

Reorder hypotheses

CUT

Introduce intermediate lemma

These correspond to **classical structural rules** of sequent calculus.
CUT is the most powerful — and the most dangerous (see Slide 100).

Move Vocabulary — Logical Rules

VERIFY

Send sub-goal to Z3

CONSTRUCT

Synthesize a witness / solution

NORMALIZE

Simplify; strip irrelevant diffs

INTRODUCE

Introduce connective (e.g. $\forall$ -intro)

ELIMINATE

Eliminate connective (e.g. $\exists$ -elim)

VERIFY bridges **semantic moves** and the **SMT layer**:
it dispatches the classified fragment to Z3.

Move Vocabulary — Geometric Moves (NEW!)

RESTRICT

Narrow to a sub-coordinate

TRANSPORT

Move evidence along a morphism

GLUE

Stitch local sections into global proof

REFINE_COVER

Improve the covering of a space

No Analogue in LEAN / Coq / F*

These moves exist because JuGeo has **geometric structure**:
coordinates, covers, restriction maps, and gluing conditions.

Move Vocabulary — Evidence Moves (NEW!)

PROMOTE_TRUST

Upgrade trust with justification

CHALLENGE

Question existing evidence

ESCALATE

Defer to oracle (lower trust)

Operating on Trust, Not Just Terms

- Traditional tactics: transform proof **terms**
- Evidence moves: transform **trust levels** and **evidence quality**
- CHALLENGE can *downgrade* trust if evidence is suspect

Move Vocabulary — Treaty Moves (NEW!)

NEGOTIATE_TREATY

Reconcile conflicts between overlapping code sections

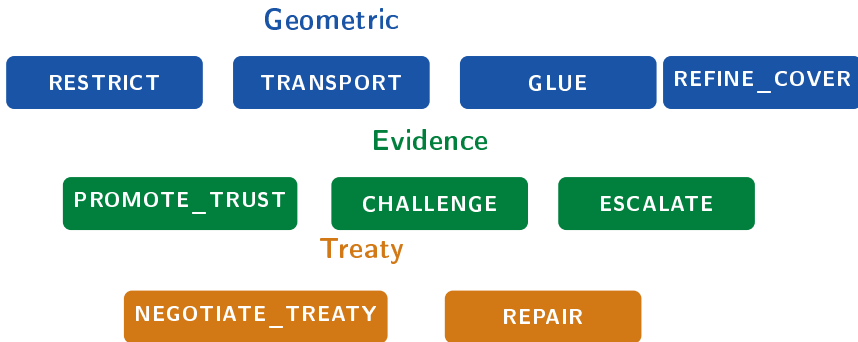
REPAIR

Fix obstructions when gluing fails

When Code Sections Overlap

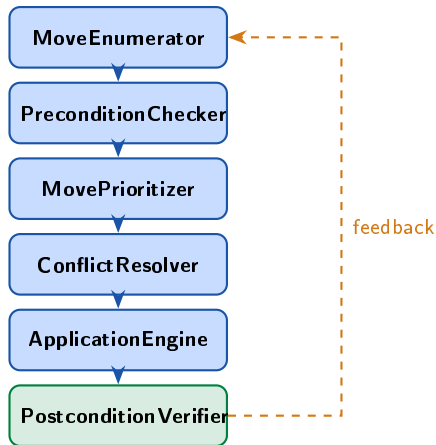
- Module A and Module B both define `validate()`
- They **disagree** on return type
- **NEGOTIATE_TREATY**: find common ground
- **REPAIR**: patch the obstruction so gluing succeeds

9 Moves With No Analogue in LEAN / Coq / F*



These **9 moves** exist because JuGeo has geometric structure, graded trust, and inter-module treaty management — concepts absent from traditional proof assistants.

Adaptive Move Selection



Not a fixed script — an **adaptive control loop**.
Failed postconditions feed back to enumeration.

Control Strategies

GREEDY

Always pick highest expected gain

LOOKAHEAD

Simulate k steps ahead, then choose

BALANCED

Trade off gain vs. cost

ADAPTIVE

Switch strategy mid-proof based on progress

More Powerful Than Meta-F* Combinators

- F*: `or_else`, `seq`, `repeat` — combinators on tactics
- JuGeo: full **strategy switching** with lookahead and cost models

Cut Elimination — A Key Theorem



Theorem (Cut Admissibility)

Every proof in JuGeo that uses **CUT** can be transformed into an equivalent proof that does **not** use CUT.

Why This Matters

- **Consistency**: no circular reasoning smuggled via cuts
- **Normalization**: every proof has a canonical form
- **Sub-formula property**: conclusions only involve sub-formulas of hypotheses
- This is JuGeo's analogue of strong normalization in type theory

What Is a Proof Obligation?

Definition

A **proof obligation** is something the system needs you (or a solver) to prove before it will trust a piece of code.

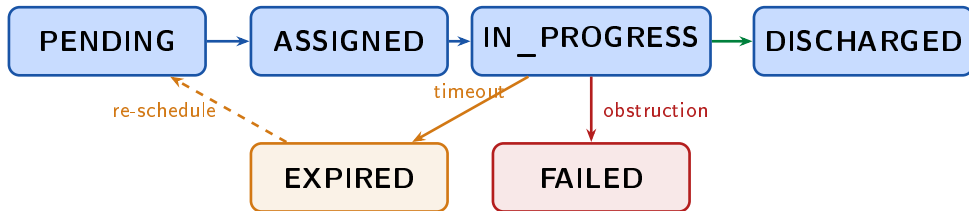
Example

```
def withdraw(balance, amount):  
    assert amount >= 0
```

Obligation: *“withdraw always receives a non-negative amount.”*

*The system **generates** obligations; backends **discharge** them.*

Obligation Lifecycle



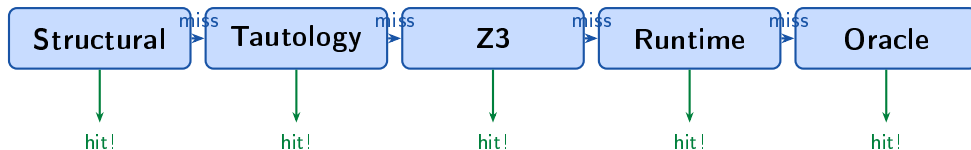
- Happy path: PENDING → ASSIGNED → IN_PROGRESS → DISCHARGED
- Failure records an **obstruction**; expiry allows **retry**

Five Discharge Backends

#	Backend	Method	Trust Level
1	Structural	Vacuous / guaranteed	VERIFIED_PROOF
2	Tautology	Pattern matching	VERIFIED_PROOF
3	Z3	Fragment-aware SMT	SOLVER_DISCHARGED
4	Runtime	Test execution	RUNTIME_WITNESSED
5	Oracle	LLM / human	COPILOT_SUGGESTED

*Higher in the table = cheaper **and** more trusted.*

Cascade Discharge



- Try the **cheapest** backend first; fall through on miss
- Each hit assigns the **appropriate trust level**
- Like a medical triage system: easy cases resolved quickly

Cohomology Classification of Discharge

H^0 — Fully discharged

H^1 — Partial gap

H^2 — Failed

$H^{-\infty}$ — Deferred to oracle

- H^0 : global section exists — obligation fully met
- H^1 : residual gap — partially discharged
- H^2 : obstruction class — proof impossible on current evidence

F* vs JuGeo — Obligation Handling

F* Pipeline

- 1 VC generated
- 2 Sent to Z3
- 3 sat / unsat / unknown

One backend. Binary outcome.
No trust annotation.

JuGeo Pipeline

- 1 VC generated
- 2 **Classified** by type
- 3 **Routed** to best backend
- 4 Multi-backend **cascade**
- 5 Trust-annotated result
- 6 Obstruction if failed

*JuGeo never says just “unknown” — it tells you **why** and **what to do next**.*

What Is an Obstruction?

Key idea

When a proof **fails**, the failure itself becomes a **first-class object**.

Traditional

- Error message printed
- Scrolls off screen
- No structure, no memory

JuGeo

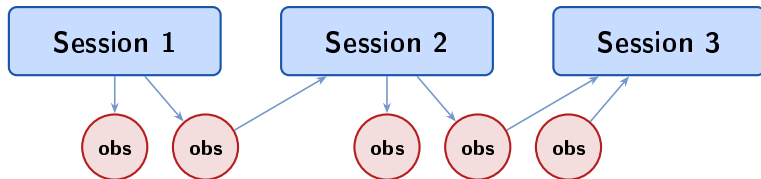
- **Obstruction** object created
- Stored in the registry
- Queryable, persistent, repairable

Not an error message — a structured mathematical record.

Anatomy of an Obstruction

obstruction_id	<code>"obs-042"</code>
coordinate	<code>where in the codebase</code>
proposition	<code>what was supposed to hold</code>
admissibility	<code>which law was violated</code>
evidence_present	<code>True / False</code>
repair_frontier	<code>concrete fix suggestions</code>
cohomology_class	<code>H0 / H1 / H2</code>
downstream	<code>obligations that depend on this</code>

Obstructions Are Persistent



Obstructions survive across sessions and accumulate in a registry.

- Unlike error messages that **scroll away**
- Obstructions **persist**, **accumulate**, and form a structured history
- You can return days later and pick up exactly where you left off

Obstructions Are Queryable

```
# All obstructions below SOLVER_DISCHARGED trust
low_trust = registry.query(
    trust_below="SOLVER_DISCHARGED"
)

# All obstructions in the auth module
auth_obs = registry.query(
    coordinate_prefix="myapp.auth"
)

# Obstructions that have repair frontiers
repairable = registry.query(
    has_repair_frontier=True
)
```

*Obstructions are a **database**, not a log file.*

Obstructions Have Repair Frontiers

What is a repair frontier?

Each obstruction carries **concrete suggestions** for how to resolve it.

Example Repair Frontier

- “Check edge case: empty list”
- “Verify base case `total=0` satisfies spec”
- “Add precondition `len(items) > 0`”

Traditional

Error: assertion failed
Opaque. Now what?

JuGeo

Structured repair steps
Actionable. Try these first.

Obstruction Example

```
Obstruction(  
  obstruction_id = "obs-spec-affine-007",  
  coordinate     = "spec.affine.program.00",  
  proposition     = "spec(solve([], -2, 3, 0), ...) == True",  
  admissibility  = "GLUING_LAW",  
  evidence       = False,  
  repair_frontier = [  
    "Check edge case: empty list",  
    "Verify base case total=0 satisfies spec",  
  ],  
  cohomology_class = "H1",  
  downstream     = ["obl-affine-012", "obl-affine-013"],  
)
```

H^1 : partial gap — evidence is
missing but a **repair path** exists.

Backpressure — When Obstructions Pile Up

OBLIGATION_OVERFLOW

INTEGRATION_LAG

TREATY_INSTABILITY

EVIDENCE_EXHAUSTION

BUDGET_CRITICAL

- Too many unresolved obligations? The system notices.
- Each family triggers a different **response strategy**
- No other proof assistant has backpressure management

Backpressure Responses

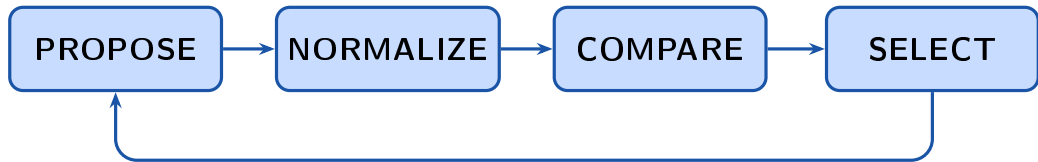
Response	What Happens
THROTTLE	Slow down: reduce obligation generation rate
PAUSE	Stop: halt verification until pressure drops
REDIRECT	Reroute: try a different backend or strategy
ESCALATE	Ask for help: flag for human / LLM review
SHED_LOAD	Drop low-priority obligations to free capacity

*Think of it as **congestion control** for proofs — the same idea that keeps the internet from collapsing.*

The Construction Loop — Overview

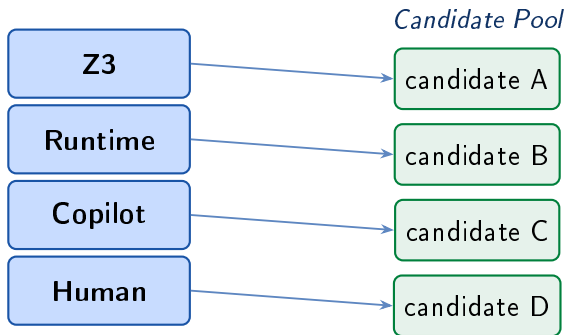
JuGeo's core verification engine

Four phases that repeat until all obligations are discharged (or deferred).



- Analogous to F*'s VC generation + discharge, but **generalized**
- Multiple candidates compete; the best one wins each round

Phase 1 — PROPOSE



- Solicit candidate solutions from **all channels** simultaneously
- Multiple candidates **compete** — no single point of failure
- Even partial solutions are welcome (they carry partial evidence)

Phase 2 — NORMALIZE

Before normalization

```
candidate A: \x -> x + 1  
candidate B: lambda y:  y + 1  
candidate C: def f(z):  return z+1
```

After normalization

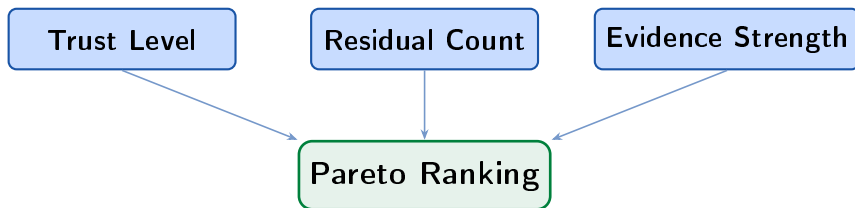
```
candidate A:  $\lambda v_0. v_0 + 1$   
candidate B:  $\lambda v_0. v_0 + 1$   
candidate C:  $\lambda v_0. v_0 + 1$ 
```

What normalization does

- **Alpha-equivalence**: rename variables to canonical forms
- **Formatting**: strip whitespace and stylistic differences
- **Desugar**: expand syntactic sugar to core forms

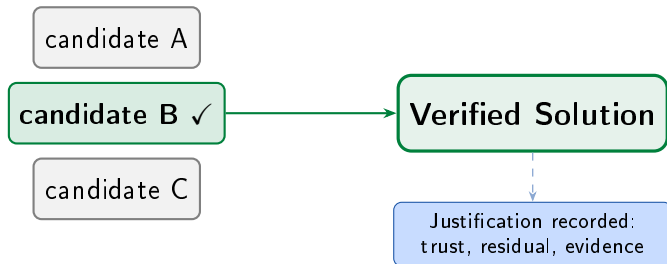
*So candidates can be **fairly compared** on substance, not style.*

Phase 3 — COMPARE



- **Trust level**: `VERIFIED_PROOF > SOLVER_DISCHARGED > RUNTIME_WITNESSED > COPILOT_SUGGESTED`
- **Residual count**: fewer unresolved obligations = better
- **Evidence strength**: how much of the spec is covered?
- Multi-criteria $\Rightarrow$ **Pareto ranking**, not a single score

Phase 4 — SELECT



- Pick the Pareto-optimal candidate
- **Explicit justification** recorded — auditable choice
- Winner becomes the verified solution for this coordinate
- Losers are kept as fallback evidence

Compression Records

Each construction round produces a delta record

A compact summary of **what changed** this round.

<code>section_deltas</code>	Which sections were updated
<code>obligation_deltas</code>	New, discharged, or expired obligations
<code>evidence_deltas</code>	New evidence collected this round
<code>obstruction_deltas</code>	Obstructions added or resolved
<code>certificate_updates</code>	Certificate version incremented

Full traceability: you can replay the entire veri-

Part V

Proof-Carrying Python

The proof ships with the code

Why Not Extraction?

- F* extracts to OCaml
- Lean extracts to C
- Coq extracts to OCaml / Haskell

The Problem

The shipped code carries **no proof metadata**.

“How do you know this is correct?”

→ You point to a **different codebase** in a **different language**.

JuGeo's Approach — Skip Extraction

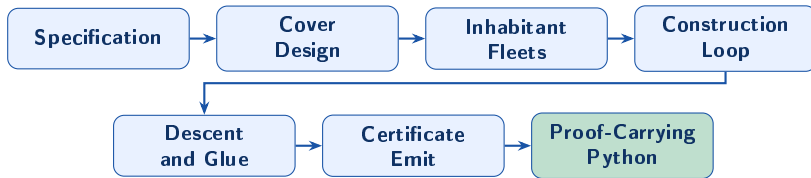
Key Idea

Generate **proof-carrying Python** directly.

- Standard Python code
- Annotated with **verification metadata**
- The proof **travels** with the code
- No separate artifact, no different language

Code + Proof = **one Python file**

The Generation Pipeline

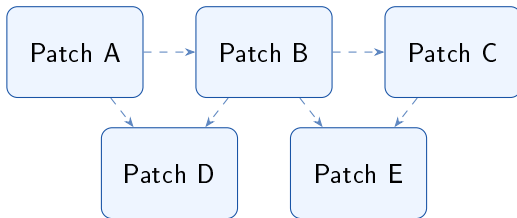


Seven stages from specification to deployable, verified Python.

Stage 1 — Cover Design

Goal

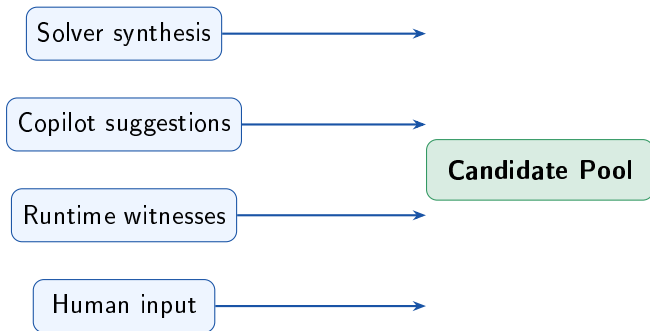
Decompose the verification goal into manageable **patches**.



Like dividing a large map into **overlapping sections** that are easier to verify individually.

Stage 2 — Inhabitant Fleets

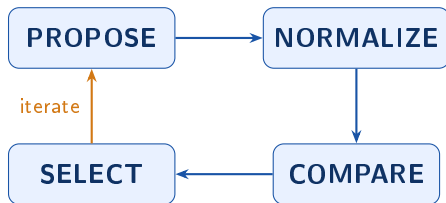
Generate **candidate implementations** from multiple sources:



Multiple candidates **compete** — best evidence wins.

Stage 3 — Construction Loop

The 4-phase loop from Part III, applied per patch:

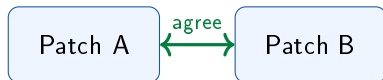


Picks the best candidate with **full trust tracking**.

Stage 4 — Descent and Glue

Overlap Check

Do all patches **agree** where they overlap?



Glues $\Rightarrow$ global proof

Conflict $\Rightarrow$ obstruction

Extract **obstructions** where patches disagree.
If everything glues, you have a **global proof**.

Stage 5 — Certificate Emit

Annotate the winning solution with **full provenance**:

Field	Content
Evidence	Which solver / runtime data supported it
Trust level	SOLVER_DISCHARGED, RUNTIME, ...
Channels	Source of each piece of evidence
Coordinate	Address in the semantic site

The certificate becomes part of the **Python file itself**.

What Proof-Carrying Python Looks Like

The scaffold starts with a **coordinate function**:

Site Address

```
def _spec_affine_program_0_coordinate():  
    return "spec.affine.program.00"
```

- The function's **address** in the semantic site
- Machine-readable, human-inspectable
- Links this file to the global verification map

Scaffold — Support Function

The **observable region** — which inputs this function covers:

Support

```
def _spec_affine_program_0_support(values):  
    support = []  
    for offset, value in enumerate(values):  
        support.append((  
            "spec.affine.program.00",  
            offset,  
            int(value)  
        ))  
    return tuple(support)
```

Returns a tuple of (coordinate, offset, value) triples.

Scaffold — Descent Profile

Overlap metadata — how this function interacts with neighbors:

Descent Profile

```
def _spec_affine_program_0_descent_profile(values):  
    base = _spec_affine_program_0_coordinate()  
    support = _spec_affine_program_0_support(values)  
    return {  
        "coordinate": base,  
        "support": support,  
        "overlaps": [], # filled by glue stage  
        "depth": 0,  
    }
```

Used by Stage 4 (Descent and Glue) to check patch compatibility.

The Implementation — Unchanged!

Your Original Python

```
def solve(values, bias, mod, keep):  
    total = 0  
    for value in values:  
        if int(value) % mod == keep:  
            total += int(value) + bias  
    return total
```

- ✓ No rewriting
- ✓ No new syntax
- ✓ Standard, importable, deployable

The Spec — Also Python

Specification

```
def spec(result, values, bias, mod, keep):  
    expected = sum(  
        int(v) + bias  
        for v in values  
        if int(v) % mod == keep  
    )  
    return isinstance(result, int) and result == expected
```

Both the **code** and the **spec** are plain Python.
Any developer can read them.

Declared Covers

The **geometric witness** — inputs where the spec must hold:

Cover Points

```
COVER = [  
  {"args": [[-2, 0, 3, 6], -2, 3, 0]},  
  {"args": [[0, 1, 2, 3, 4, 5], -2, 3, 0]},  
  {"args": [[10, 20, 30], 0, 5, 0]},  
  {"args": [[], 1, 2, 1]},  
  {"args": [[7], 3, 7, 0]},  
  {"args": [[0, 0, 0], 0, 1, 0]},  
  {"args": [[1, -1, 1, -1], 10, 2, 1]},  
  {"args": [[100], -100, 1, 0]},  
  {"args": [[3, 6, 9, 12], 1, 3, 0]},  
  {"args": [[2, 4, 6, 8], 0, 2, 0]},  
]
```

The Resulting Judgment

coordinate	"spec.affine.program.00"
proposition	solve satisfies spec on COVER
evidence	solver + runtime (10/10 cover points pass)
obligations	[] — none remaining
obstructions	[] — none found
trust	SOLVER_DISCHARGED
provenance	z3 solver × 10 inputs, runtime witness × 10 inputs

When It Fails — Obstruction Example

Cover point 5 fails. What do we get?

coordinate

```
"spec.affine.program.00"
```

proposition

```
spec(solve(*args), *args) for cover point 5
```

actual

```
solve([0,0,0], 0, 1, 0) returned 1 (expected 0)
```

repair

```
Frontier: re-check mod == 0 guard, int() cast
```

JuGeo

Structured **Obstruction** with
coordinate, proposition, repair frontier.

F*

(error) Unknown assertion failed.

No repair guidance.

The Four Proof Modes

- 1 Spec Satisfaction**
Does program satisfy spec on cover?
- 2 Equivalence**
Are two programs equal on cover?
- 3 Bug Detection**
Does program have known bug patterns?
- 4 Relational Refinement**
Compute a refined version

Bug Detection — Six Classes

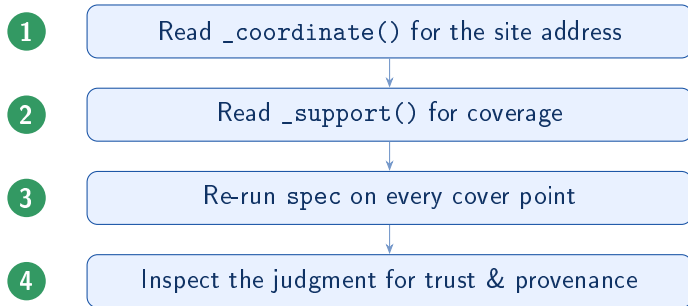
Each detected as a **structured obstruction**, not just a warning.

Bug Class	What It Catches
<code>bare-except</code>	Catches <code>except:</code> without a type
<code>identity-literal</code>	Compares with <code>is</code> to a literal
<code>late-binding-closure</code>	Closure captures loop variable by ref
<code>mutable-default</code>	Mutable default argument (<code>def f(x=[])</code>)
<code>open-without-close</code>	File handle never closed
<code>shadow-builtin</code>	Local name shadows a builtin

Every detection produces an **Obstruction** with coordinate, evidence, and **repair frontier**.

The Proof Ships With The Code

Any verifier can replay the proof from the Python file alone:



No separate artifact. No different language.

The proof is the code.

Part VI

The Same Proof — Three Ways

Lean, F^ , and JuGeo walk into a proof obligation...*

The Challenge

Prove this function correct

Given values, bias, mod, keep, compute:

$$\sum_{\substack{v \in \text{values} \\ v \bmod \text{mod} = \text{keep}}} (v + \text{bias})$$

```
def solve(values, bias, mod, keep):  
    total = 0  
    for v in values:  
        if int(v) % mod == keep:  
            total += int(v) + bias  
    return total
```

Three languages. Three proof styles. One verdict.

In Lean 4 — Step 1: Rewrite Your Code

New language required

You must rewrite the Python in Lean's functional style.

```
def spec (values : List Int) (bias mod keep : Int) : Int :=  
  (values.filter (fun v => v % mod == keep)).foldl  
    (fun acc v => acc + v + bias) 0
```

- Learn Lean syntax, dependent types, universe levels
- Maintain **two codebases**: Python + Lean
- No guarantee the Lean code matches the Python

In Lean 4 — Step 2: Write a Tactic Proof

```
theorem solve_eq_spec
  (values : List Int) (bias mod keep : Int)
  : solve values bias mod keep
    = spec values bias mod keep := by
  unfold solve spec
  induction values with
  | nil => simp
  | cons v vs ih => simp; split; omega
```

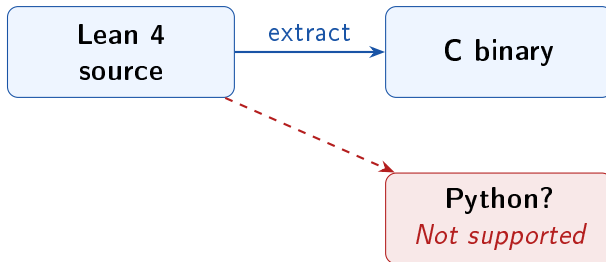
Fragile

Change one line of code $\Rightarrow$
proof breaks entirely

Manual

Every branch, every case,
written by hand

In Lean 4 — Step 3: Extract to C



- Output carries **no proof trace**
- If you want Python — tough luck
- Proof checked at compile time, invisible at runtime

In F* — Step 1: Rewrite Your Code

Another new language

F*'s ML-like syntax with totality annotations.

```
let rec solve (values: list int) (bias mod_ keep: int)
  : Tot int (decreases values) =
  match values with
  | [] -> 0
  | v :: vs ->
    if v % mod_ = keep
    then (v + bias) + solve vs bias mod_ keep
    else solve vs bias mod_ keep
```

- Must learn F* syntax *and* totality annotations
- Still maintaining **two codebases**

In F* — Step 2: Write a Recursive Lemma

```
let rec solve_correct (values: list int)
  (bias mod_ keep: int)
  : Lemma (ensures solve values bias mod_ keep
    = expected_total values bias mod_ keep)
    (decreases values) =
  match values with
  | [] -> ()
  | _ :: vs -> solve_correct vs bias mod_ keep
```

Pro

Z3 auto-discharges arithmetic —
less manual than Lean!

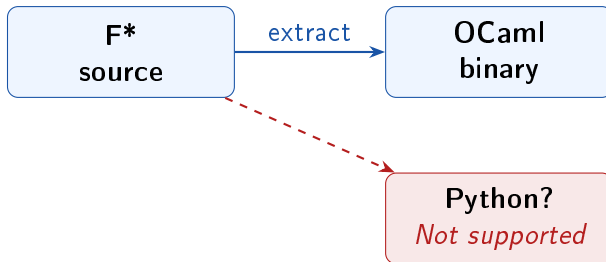
Con

When Z3 times out:

"Unknown."

No explanation. No fix.

In F* — Step 3: Extract to OCaml



- No provenance in extracted output
- No Python target
- Same pattern: proof stays behind, code ships alone

In JuGeo — Step 1: Write Python

No new language. This is your real code.

```
def solve(values, bias, mod, keep):  
    total = 0  
    for value in values:  
        if int(value) % mod == keep:  
            total += int(value) + bias  
    return total
```

That's it. Step 1 is done.

In JuGeo — Step 2: Write a Spec (Also Python)

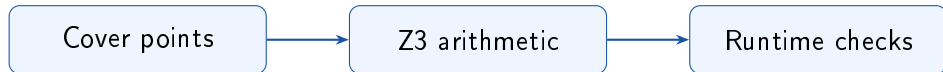
```
def spec(result, values, bias, mod, keep):  
    return isinstance(result, int) and \  
        result == expected_total(values, bias, mod, keep)
```

```
def expected_total(values, bias, mod, keep):  
    return sum(int(v) + bias  
        for v in values  
        if int(v) % mod == keep)
```

- Spec is a **plain Python predicate**
- Returns True when result is correct
- No tactics, no lemmas, no new syntax

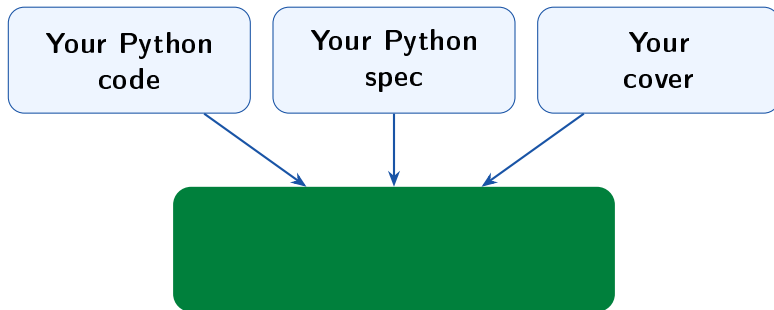
In JuGeo — Step 3: Declare a Cover

```
cover = [  
  {"values": [],      "bias": 0, "mod": 2, "keep": 0},  
  {"values": [4,7,10], "bias": 1, "mod": 3, "keep": 1},  
  {"values": [1,2,3],  "bias": 5, "mod": 1, "keep": 0},  
  # ... more points to span the input space  
]
```



JuGeo does the rest: Z3 for arithmetic, runtime for edge cases, scaffold for **provenance**.

In JuGeo — The Result



- The **same code** you wrote, now annotated
- Verification metadata travels with the artifact
- Any downstream verifier can re-check — in Python

Side-by-Side Summary

	CCC		
	Lean 4	F*	JuGeo
Write in	Lean	F*	Python
Ship as	C	OCaml	Python (same!)
Proof effort	Manual tactics	Recursive lemma	Cover + spec
On failure	“tactic failed”	“Unknown”	Structured ob- struction
Trust model	Binary	Binary	7-level algebra

Why JuGeo Is Easier — Reason 1

You don't rewrite your code.

- **No second language** to learn
- No two codebases to keep in sync
- The code under verification *is* the code you ship

You don't write tactic proofs.

- Declare the **cover** and the **spec**
- The semantic-move engine does the proof search
- Arithmetic goes to Z3; edge cases go to runtime
- You describe *what* to verify, not *how* to prove it

Why JuGeo Is Easier — Reason 3

Failure is actionable.

Lean / F*

tactic failed

Unknown

Now what?

JuGeo

Structured **obstruction**
with a **repair frontier**:

what failed and *how* to fix it

Why JuGeo Is Easier — Reason 4

Mixed evidence is natural.

Z3 proved **8/10** points

Tests covered **2** edge cases

Honest account — not all-or-nothing

Why JuGeo Is Easier — Reason 5

The proof ships with the code.



- Deployed Python artifact is **self-describing**
- Any verifier can re-check without leaving Python
- No separate proof artifact to lose or mismatch

What F* Can't Express — Context Managers + Async

```
async def fetch(url):  
    async with aiohttp.ClientSession() as session:  
        async with session.get(url) as response:  
            return await response.json()
```

F* would need

- Custom effect for aiohttp
- Model for context managers
- Model for async/await
- Bindings for every library

JuGeo

- Models it **natively**
- Python is the language
- Runtime channel covers actual I/O behaviour
- No bindings needed

**Write Python. Get verified
Python. Ship verified Python.**

All in one language, with honest trust accounting.



Part VII

Under the Hood — Architecture

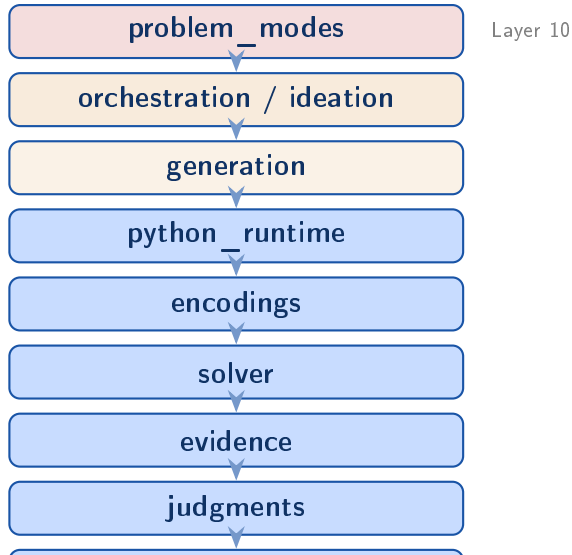
960 modules, 10 layers, one dependency DAG

~960

Python modules

- Organized in a strict **dependency DAG** (directed acyclic graph)
- Each layer builds only on the layers below it
- No circular imports, no upward calls
- Like the layers of an operating system

The Architecture DAG — Overview



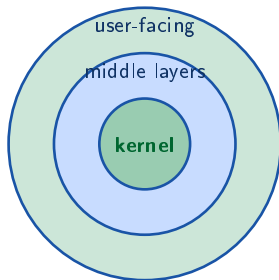
Layer 1 — Kernel

What It Does

- Core services and lifecycle management
- Authority and trust anchoring
- The **trusted computing base**

Analogy

Like an OS kernel — **small, trusted**, and everything else builds on it.



Layer 2 — Geometry

Components

- **Sites** — the topological space of your codebase
- **Coordinates** — precise addresses (`module.function.line`)
- **Covers** — collections of inputs that span the behavior space
- **Descent** — 4 strategies for gluing local proofs into global ones
- **Hypercovers** and **supports**

*The mathematical foundation
— the “map” of your codebase*

Layer 3 — Judgments

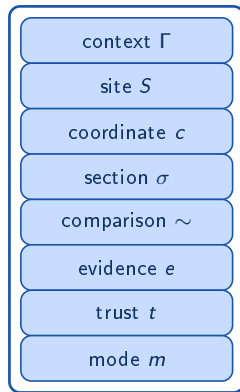
The 8-Tuple Algebra

- Contexts
- Sections
- Comparisons
- Judgment terms

Where **claims about code** live.

Every verification result is expressed as a judgment.

Judgment



Layer 4 — Evidence

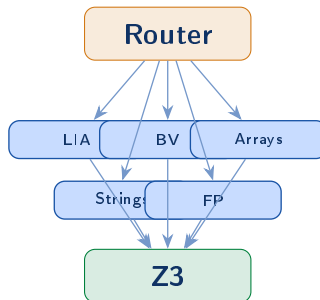
What It Provides

- **Trust algebra** — combine, compare, and degrade trust values
- **6+ evidence channels**: Z3, testing, AI, manual, runtime, static
- **Certificates** — unforgeable proofs of evidence origin
- **Provenance** — complete audit trail for every claim
- **Manifests** — packaged evidence bundles

The “courtroom” where evidence is weighed and tracked

Z3 Integration

- **13 theory fragments** (arithmetic, arrays, bitvectors, ...)
- **5 routing strategies** to pick the best fragment
- Session pool for parallel queries
- Countermodel extraction on failure



18 sub-packages — translating Python into logic

- Deduction rules
- Refinement types
- Path conditions
- Heap summaries
- Theorem schemas
- Text encodings
- Tensor encodings
- Effect encodings
- Inductive encodings
- ... and 8 more

*The largest layer — the bridge from
Python semantics to Z3 formulas*

Layer 7 — Python Runtime

JuGeo's Model of Python Execution

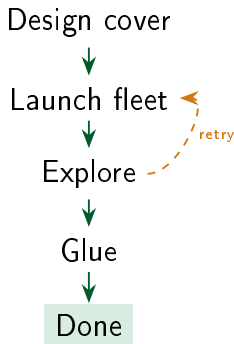
- **Scope & state** — variable bindings, mutation tracking
- **Exceptions** — try/except/finally flow
- **Async** — coroutines, await, event loops
- **Generators** — yield, send, close
- **Context managers** — with/as protocol
- **Metaobject protocol** — `__getattr__`, descriptors, metaclasses
- **Program loading** — imports, `__init__`, `sys.path`

*Not a toy subset — the **real** Python semantics*

Layer 8 — Generation

The Proof Search Engine

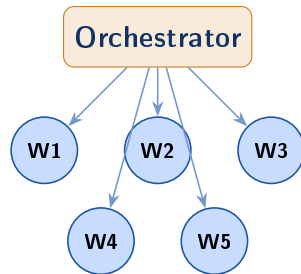
- **Cover design** — choosing the right decomposition
- **Construction loops** — iterative refinement
- **Inhabitant fleets** — parallel proof candidates
- **State space exploration**
- **Backpressure** — throttling runaway search
- **Replay & gluing** — caching and composition



Layer 9 — Orchestration

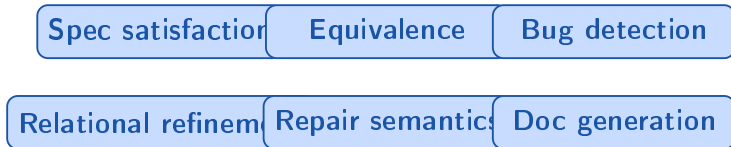
The Conductor

- **Synthesis orchestrator** — top-level control
- **Fleet management** — allocating proof workers
- **Semantic control** — what to prove next
- **Move selection** — game-tree search over tactics
- **Mixed-evidence routing**
- **Frontier objectives** — prioritizing open goals



Coordinates a fleet of proof workers

Layer 10 — Problem Modes



- Each mode is a user-facing **verification goal**
- Same architecture underneath — different question at the top
- You pick the mode; JuGeo picks the strategy

The CLI Interface

Simple Entry Points for Complex Verification

```
# Prove a specification holds
$ jugeo prove myapp.auth

# Find bugs automatically
$ jugeo bugs myapp.database

# Check two implementations are equivalent
$ jugeo equiv myapp.sort_v1 myapp.sort_v2

# Encode Python into logic (inspect)
$ jugeo encode myapp.parser

# Run descent (glue local proofs)
$ jugeo descend myapp

# Load and analyze a codebase
$ jugeo load myproject/
```

Seven one-function entry points

One Import, One Call

```
from jugeo import prove, bugs, equiv
from jugeo import ideate, carry, spec, research

# Prove a function meets its spec
result = prove("myapp.auth.validate_token")

# Find bugs in a module
issues = bugs("myapp.database")

# Check equivalence of two implementations
ok = equiv("myapp.sort_v1", "myapp.sort_v2")
```

One function per capability. Designed for Python developers.

70+

experiments

- Site scaling
- Judgment algebra
- Descent obstructions
- Cover design
- Bug detection
- Heap aliasing
- Countermodel extraction
- ...and many more

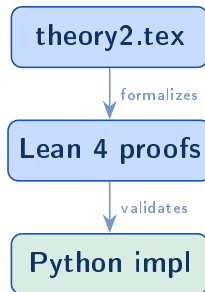
All in `experiments/` — reproducible with one command

Formal Theory

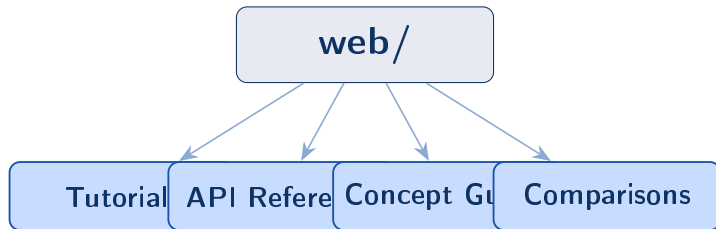
- **67-chapter** formal theory document
- Covers: sites, judgments, descent, trust algebra, cut elimination, effects
- Written in $\text{\LaTeX}$ (`theory2.tex`)

Lean 4 Formalizations

Machine-checked proofs in `proofs/lean/`
Core theorems verified independently.



The Web Interface



- Full documentation site in the project's `web/` directory
- Step-by-step tutorials for new users
- API reference auto-generated from docstrings
- Comparison pages: JuGeo vs. LEAN, Coq, F*, Dafny

Testing Infrastructure

Scale

- 300+ **benchmark cases** across 8 families
- Integration tests for CLI robustness
- Directed research tests
- Research orchestration tests
- Comprehensive pytest suite

8 Families

Unit tests

Integration tests

CLI tests

Benchmark cases

Research tests

Orchestration

Regression

End-to-end

Deduction Rules — First-Class Inference

Structural Rules

- Weakening
- Contraction
- Exchange
- Cut

Semantic Rules

- Introduction
- Elimination
- Computation

Judgment Transitions

Rules that move between judgment forms:

- Verification $\rightarrow$ synthesis
- Local $\rightarrow$ global (descent)
- Single $\rightarrow$ relational

All rules are **first-class Python objects** —
inspect, compose,
and replay them

Part IX

The Geometry of Ideation

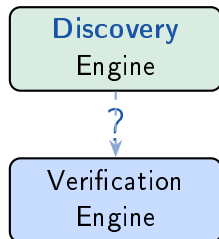
How JuGeo discovers what to prove next

So far, JuGeo can:

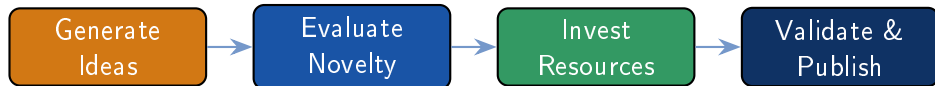
- Model code as a geometric site
- Attach evidence with trust levels
- Detect and repair failures
- Ship proof-carrying Python

But **who decides what to verify?**

- Which functions need specs?
- What invariants might hold?
- What ideas are worth pursuing?

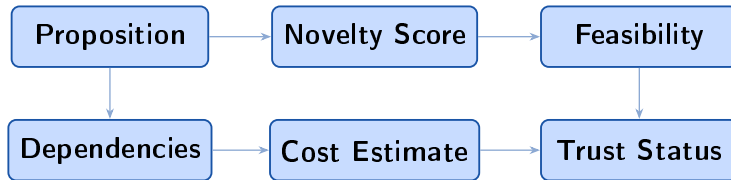


Analogy: Running a Research Lab



- A **research lab** doesn't just execute experiments blindly
- It **generates** candidate ideas, **evaluates** their novelty, **invests** limited budget wisely, and **validates** results
- JuGeo's **ideation engine** does the same thing for code verification and theorem discovery
- It treats **ideas as structured objects** — not strings, not hunches

Ideas as Structured Objects



An **Idea** in JuGeo is not just a string — it's a typed object:

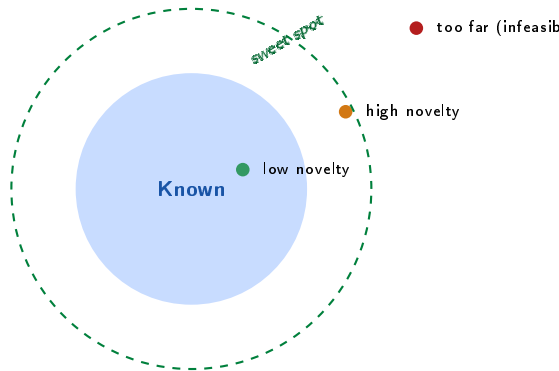
- **Proposition:** What is being claimed?
- **Novelty:** How different is this from what we already know?
- **Feasibility:** Can we actually prove or validate it?
- **Dependencies:** What must be true first?
- **Cost:** How much effort would it take?
- **Trust status:** Proposed? Partially validated? Fully proven?

The Question

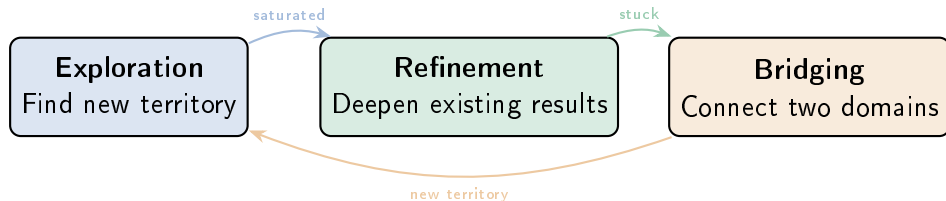
Is this idea **actually new**, or just a restatement of something we already know?

JuGeo measures novelty as **semantic distance**:

- Maintain a **theorem portfolio** of known results
- Score each candidate against the portfolio
- High novelty = far from existing knowledge
- But novelty alone isn't enough — the idea must also be **useful**



Regimes — Different Modes of Discovery



- Like a GPS with multiple route modes, JuGeo picks the right **discovery regime** for the current situation
- **Exploration**: cast a wide net when the search space is open
- **Refinement**: go deeper when existing results can be strengthened
- **Bridging**: connect two domains when each is stuck alone
- Regimes transition automatically based on progress and diminishing returns

Analogy Transport



- A theorem about sorting can sometimes be **transported** to a structurally similar domain
- “Merge preserves order” $\longrightarrow$ “Merge join preserves key uniqueness”
- JuGeo finds these structural similarities and transports ideas across domains — **with trust attenuation**
- The transported idea starts at PROPOSED trust (never silently promoted to VERIFIED)
- This is how discoveries in one codebase can accelerate verification of another

The Discovery Pipeline



Generate Produce candidate ideas from code analysis, analogy, or AI suggestion

Score Measure novelty against the known portfolio

Classify What kind of idea is it? Invariant? Equivalence? Bug pattern? New concept?

Validate Test the idea — run experiments, check with the solver, attempt a proof

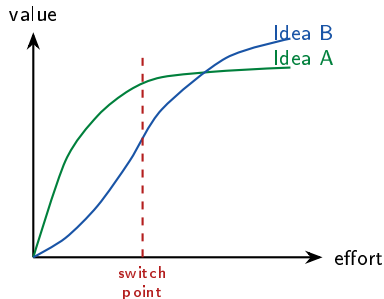
Promote If validated, inject into JuGeo's judgment sheaf as a coordinate with trust and evidence

The Problem

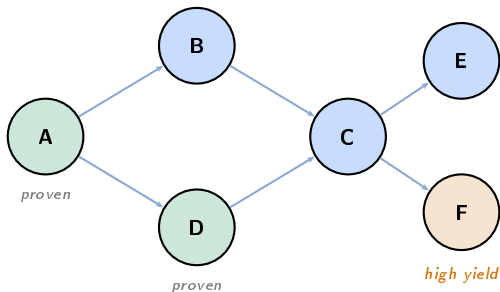
We have **limited budget** (time, compute, human attention) and **many candidate ideas**. Which should we pursue?

JuGeo treats this as an **investment problem**:

- Each idea has an expected **yield** (how much verified coverage it adds)
- Each idea has a **cost** (solver time, human effort, risk of failure)
- **Diminishing returns** mean switching topics is sometimes better

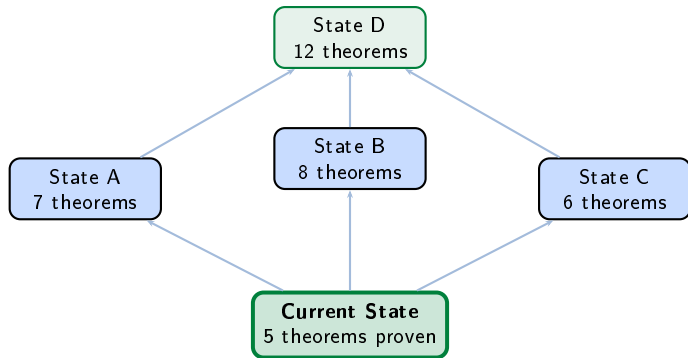


Theorem Ecologies — Ideas That Help Each Other



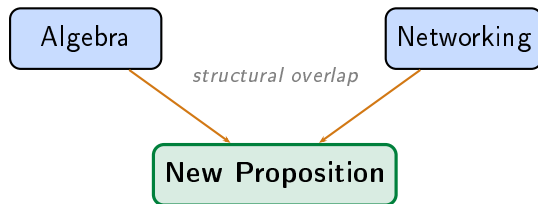
- Theorems form an **ecology**: proving A makes B easier, which unlocks C, which enables F
- JuGeo tracks these **compounding effects**
- A cheap theorem that unlocks expensive ones is worth more than its individual yield
- The economics layer optimizes for **portfolio value**, not just individual theorems

Semantic Futures — Where Could We Go?



- JuGeo models the space of **reachable future states**
- Each future state represents a set of proven theorems
- The ideation engine asks: “which futures are most valuable and most reachable?”
- Uses beam search, diversity constraints, and budget allocation to navigate toward high-value futures

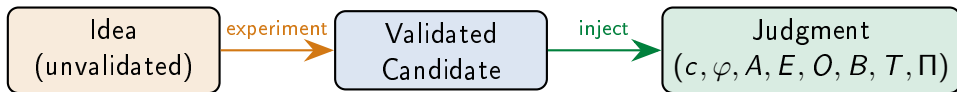
The Synthesis Frontier



The **synthesis frontier** combines ideas from different fields:

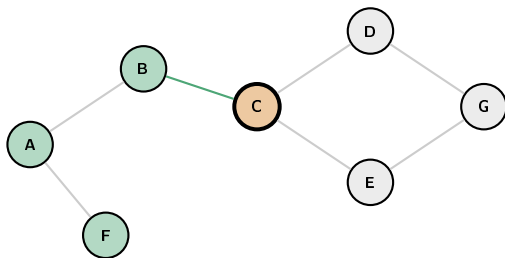
- JuGeo maintains a catalog of **128 knowledge fields**
- It finds **structural overlaps** between field pairs
- Generates cross-domain propositions via **metaphor and pattern matching**
- Runs a **tournament** to select the most promising candidates
- Winners enter the judgment sheaf at PROPOSAL trust

From Idea to Judgment



- Every validated idea becomes a **judgment 8-tuple**
- Gets a coordinate in the site, a proposition, initial evidence, obligations, and PROPOSAL-level trust
- From there, the normal verification pipeline takes over: descent, evidence routing, trust promotion
- **Trust discipline is preserved**: an idea from AI starts low and must earn its way up

Theory Navigation — Exploring the Map



- The entire space of ideas forms a **navigable theory graph**
- **Green** = explored, **orange** = current position, **gray** = unexplored
- JuGeo tracks maturity, coverage, and frontier novelty
- Navigation strategies: depth-first (specialize), breadth-first (survey), purpose-directed (find specific things)
- This is how JuGeo plans an entire **research programme**, not just individual proofs

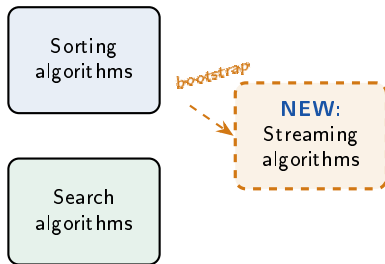
Sometimes the **existing categories** aren't enough.

Example

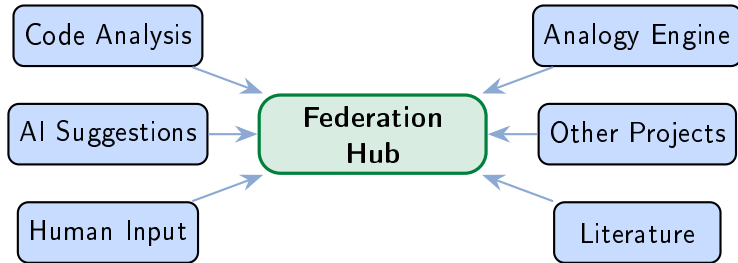
You've been verifying sorting algorithms. Then you encounter a **streaming sort** that doesn't fit any existing pattern.

JuGeo can detect when it encounters something that doesn't fit existing kinds and **bootstrap a new regime**:

- Detect anomalous obstruction patterns
- Identify new structural regularities
- Create a new “chart” in the theory atlas
- Populate it with initial theorems



Federated Discovery



- Ideas come from **multiple sources** with different trust levels
- The **federation hub** reconciles, deduplicates, and ranks them
- Trust is preserved: AI suggestions $\neq$ formal analysis
- Conflicts between sources are resolved by consensus or escalation

Why Ideation Matters

Without Ideation

- Human must decide what to verify
- No systematic exploration
- Missed opportunities
- Verification is **reactive**

With Ideation

- System discovers what to verify
- Novelty-guided exploration
- Cross-domain transport of insights
- Verification is **proactive**

*JuGeo doesn't just check your code — it **discovers things about your code** you didn't know to ask.*

Ideation Summary

Component	What It Does
Ideas	Structured proposals with novelty, cost, and trust
Novelty Search	Purpose-conditioned distance from known theorems
Regimes	Exploration / refinement / bridging modes
Analogy Transport	Move ideas across domains, preserving structure
Discovery Pipeline	Generate → score → classify → validate → promote
Theorem Economics	Budget-optimal investment across candidate ideas
Theorem Ecologies	Track compounding effects between theorems
Semantic Futures	Model and navigate reachable future states
Synthesis Frontier	Cross-domain invention via 128-field catalog
Kind Discovery	Bootstrap new conceptual regimes when needed
Federation	Reconcile ideas from multiple sources with trust

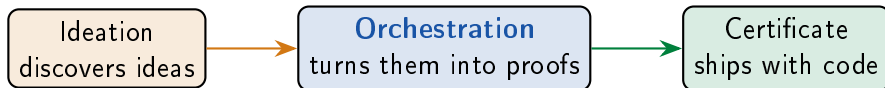
Next: **The Geometry of Orchestration** — coordinating the whole

Part X

The Geometry of Orchestration

How JuGeo coordinates proof, evidence, and repair

From Ideas to Action



- Ideation generates **what** to prove
- Orchestration decides **how** to prove it:
 - Which solver? Which evidence channel?
 - How much budget? When to give up?
 - What if two strategies disagree?
- Think of it as an **air traffic controller** for proof search

Analogy: Air Traffic Control

Air Traffic Control

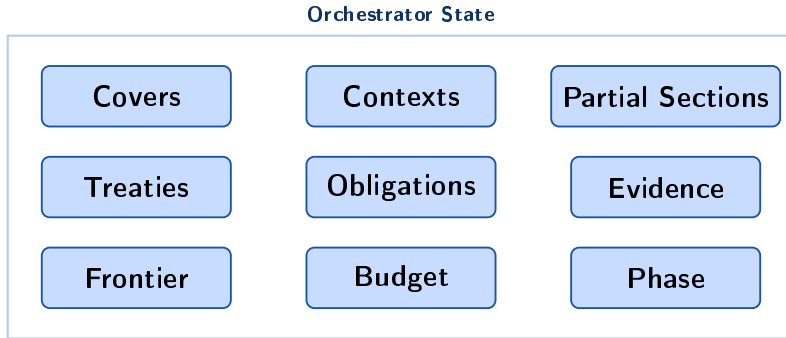
- Many planes (flights) at once
- Limited runway slots
- Must avoid collisions
- Priorities: emergencies first
- Reroute when weather changes
- Never lose track of anyone

JuGeo Orchestration

- Many proof tasks at once
- Limited compute budget
- Must avoid contradictions
- Priorities: high-trust first
- Reroute when solver stalls
- Never lose provenance

Orchestration is a control problem — managing multiple concurrent activities under resource constraints, with safety guarantees.

The Orchestration State



At any moment, the orchestrator tracks:

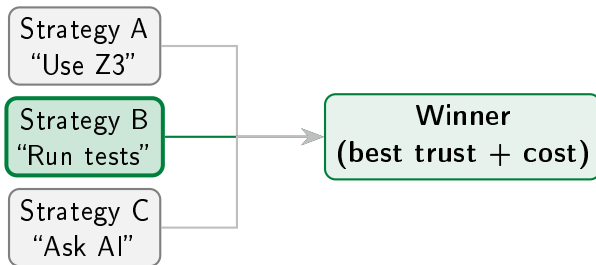
- What's **covered**, what's **partially proven**, what **obligations** remain
- What **evidence** has been collected and at what **trust**
- How much **budget** is left, what **phase** we're in
- What **treaties** (agreements between subsystems) are active

Semantic Moves — The Available Actions

Move Category	Examples
Structural	Restrict to sub-coordinate, weaken a claim, split a goal
Logical	Apply an inference rule, introduce a lemma, simplify
Geometric	Refine a cover, attempt descent, compute overlaps
Evidence	Query Z3, run a test, ask the AI oracle
Treaty	Propose an interface agreement, challenge a claim

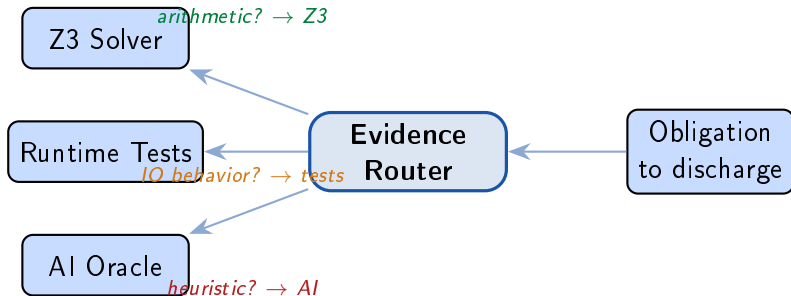
- At each step, the orchestrator has a **menu of possible moves**
- Each move has a **precondition** (what must be true to apply it) and an **effect** (how it changes the state)
- The control law picks the **best move** given the current state, budget, and objectives

Fleet Competition — Let Strategies Compete



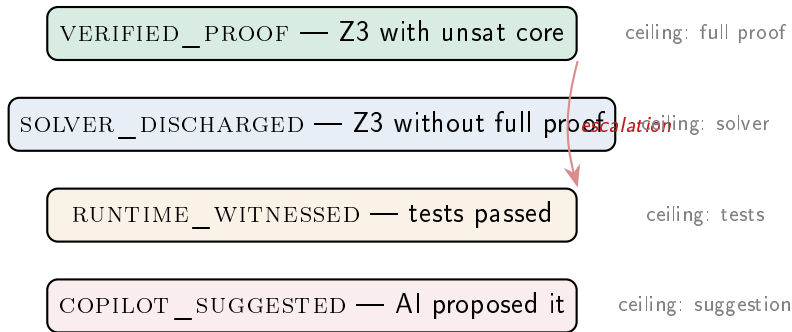
- Multiple **fleet members** propose solutions simultaneously
- Each bid includes: evidence, trust level, cost, and residual obligations
- A **bid evaluator** picks the winner using multi-criteria ranking
- Losers are archived as **fallback evidence** (not discarded)
- This is **competitive search**: the best approach wins on merit

Evidence Routing — Send It to the Right Place



- Not every obligation should go to the same solver
- The **evidence router** classifies each obligation and sends it to the best channel
- Routing is **trust-aware**: each channel has a trust ceiling
- If Z3 times out, the router **escalates** to the next channel

Trust-Aware Routing



- Each evidence channel has a **trust ceiling**
- AI cannot produce VERIFIED_PROOF-level evidence, period
- When a channel fails, evidence is routed down the trust hierarchy
- The final trust level is **honest**: it reflects the weakest link

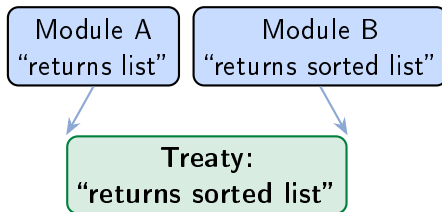
The Problem

Two subsystems make **overlapping claims** that don't quite agree.

Example

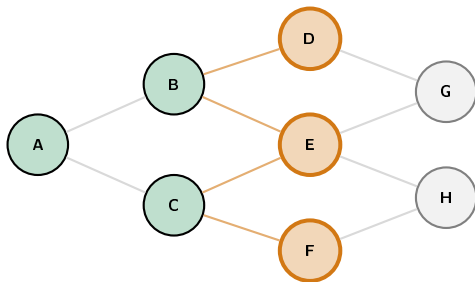
- Module A: “process() returns a list”
- Module B: “process() returns a sorted list”

These aren't contradictions — B is stronger than A. Can they agree on an **interface**?



- Treaties are **negotiated interface agreements**
- They formalize how overlapping local claims combine

Frontier Search — Exploring Proof Space

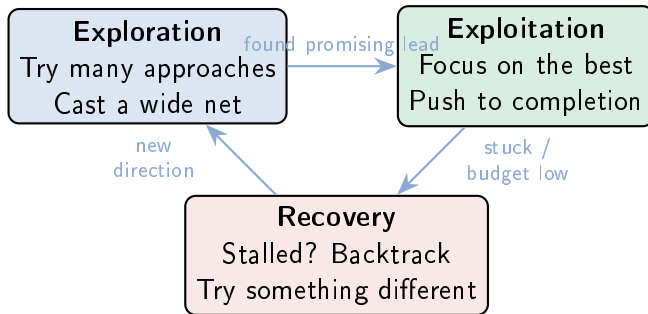


Each **frontier node** carries predictions:

- **Closure gain**: how much closer to a complete proof?
- **Stability**: will this move make things better or worse?
- **Cost**: time, solver queries, human attention
- **Diversity**: are we exploring enough different approaches?

The frontier scorer **ranks** nodes; the controller picks the best.

Phases — Exploration, Exploitation, Recovery



- **Phase transitions** are triggered by signals: progress rate, budget remaining, stall duration
- Like a chess engine switching from opening theory to endgame tactics
- The system detects **when** to switch, not just **what** to do

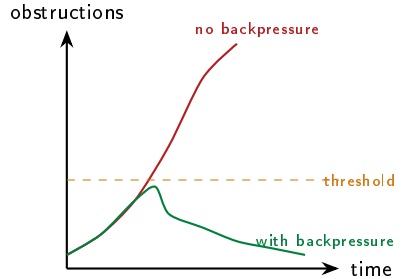
What happens when too many obstructions pile up?

Without backpressure

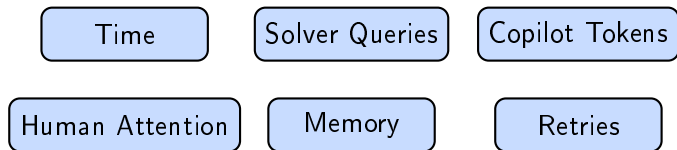
The system tries harder, creates more obligations, generates more obstructions — a **death spiral** of wasted effort.

With backpressure

The system **pauses** proof production, **repairs** the worst obstructions, then **resumes** when healthy.



Budgets — Every Resource Is Finite



- JuGeo tracks budgets across **multiple dimensions**
- Invariant: **spent** + **reserved** $\leq$ **total** always holds
- Budget enforcement prevents runaway proof searches
- Allocation is **adaptive**: spend more on high-value tasks, less on diminishing returns
- Alerts fire when approaching limits; the controller adjusts strategy

Control Laws — How Decisions Are Made

Control Law	Strategy	When to use
Greedy	Pick the locally best move	Quick wins, simple goals
Lookahead	Simulate k steps ahead	Complex proofs, traps
Balanced	Spread effort across goals	Many open obligations
Adaptive	Switch law based on feedback	Large, varied tasks

- Each control law is a **mathematical function** from state $\rightarrow$ move
- **Adaptive control** monitors progress and switches laws when the current one stalls
- This is formal control theory applied to proof search — with convergence guarantees

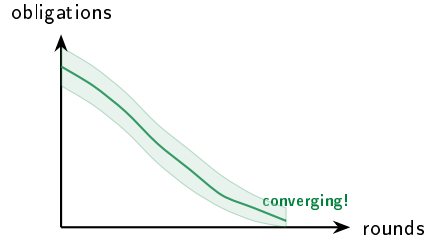
JuGeo monitors **convergence** in real time:

- **Obligation count**: going down?
- **Evidence coverage**: going up?
- **Trust levels**: improving?
- **Obstruction count**: shrinking?

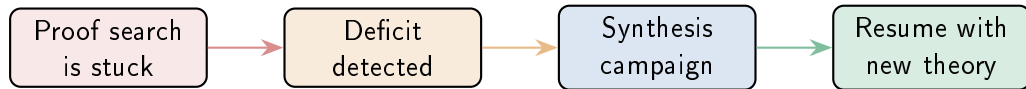
If all metrics improve → **converging**.

If metrics stall → **phase switch**.

If metrics worsen → **backtrack**.



Synthesis Fallback — When Nothing Works



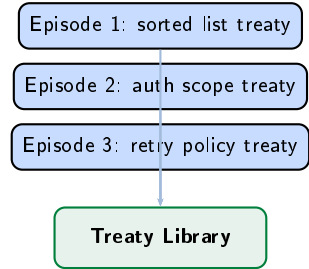
- Sometimes the current theory is **insufficient** — no combination of existing moves can close the proof
- The **synthesis orchestrator** detects this “theory deficit”
- It triggers a **synthesis campaign**: asks the ideation engine for new lemmas, analogies, or structural insights
- The new theory is injected and proof search resumes
- This is the **ideation-orchestration loop**: each feeds the other

Every negotiation produces **semantic capital**:

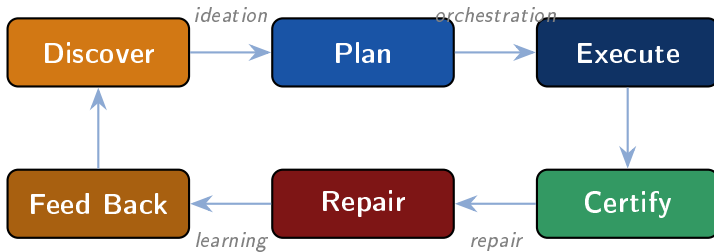
- What interface agreements worked?
- What compromise strategies resolved deadlocks?
- What treaty patterns recur across projects?

JuGeo **archives** successful treaties so future verifications can reuse them:

- Faster convergence on similar codebases
- Accumulated institutional knowledge
- Replay and audit of past decisions



The Full Loop



- **Discover** → **Plan** → **Execute** → **Certify** → **Repair** → **Feed Back** → repeat
- Ideation and orchestration form a **continuous loop**
- Each cycle improves the system's knowledge and capability
- This is JuGeo's **cyclic maturity model**: the system gets smarter with every verification run

Orchestration vs. Traditional Proof Search

Aspect	Traditional	JuGeo Orchestration
Strategy	Fixed tactic script	Adaptive control law
Evidence	Single channel (solver)	Multi-channel + routing
Competition	None	Fleet bidding
Failure	Abort	Obstruction + repair
Budget	Timeout	Multi-dimensional budget
Memory	None	Treaty archive
Phases	None	Explore / exploit / recover
Feedback	None	Synthesis fallback loop

Traditional proof search is a **single-threaded script**.

JuGeo orchestration is a **multi-strategy, budget-aware, self-correcting control system**.

Orchestration Summary

Component	What It Does
Semantic Moves	Menu of available proof actions
Fleet Competition	Multiple strategies compete on merit
Evidence Routing	Send obligations to the right channel
Trust-Aware Routing	Respect trust ceilings per channel
Treaty Negotiation	Agree on interfaces between subsystems
Frontier Search	Explore proof space systematically
Phase Management	Switch between exploration, exploitation, recovery
Backpressure	Pause when obstructions accumulate
Budget Tracking	Multi-dimensional resource management
Control Laws	Mathematical decision functions
Convergence Monitor	Are we making progress?
Synthesis Fallback	Create new theory when stuck
Treaty Memory	Learn from past negotiations

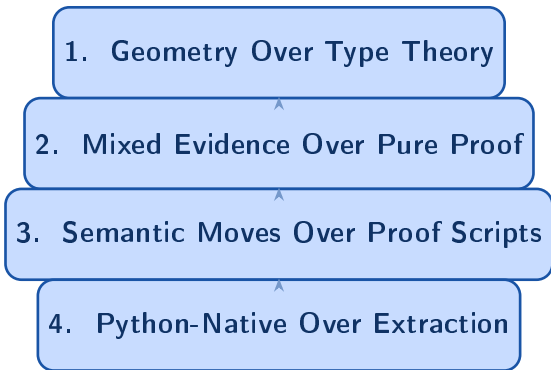
Part VIII

The Big Picture

Vision, comparison, and where we go from here

The Four Bets

JuGeo stakes its design on **four fundamental bets**:



Each bet rejects a conventional trade-off

Bet 1 — Geometry Over Type Theory

Conventional

- Proofs are **terms**
- Type-check: pass / fail
- Failure = opaque error

JuGeo

- Proofs are **sections over sites**
- Local claims glue globally
- Failure = structured obstruction

Local claim

◀ Descent / glue

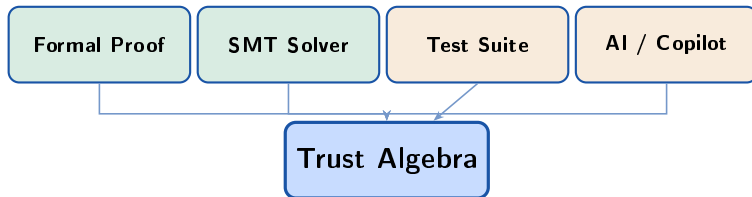
◀ Global guarantee

Bet 2 — Mixed Evidence Over Pure Proof

The Problem with Binary

Traditional systems: either you have a **proof** or you have **nothing**.

Real projects have partial evidence everywhere.



JuGeo's **trust algebra** combines all channels honestly — no pretending.

Bet 3 — Semantic Moves Over Proof Scripts

Proof Scripts

- apply, rewrite, simp
- Syntax-level manipulation
- User picks every tactic

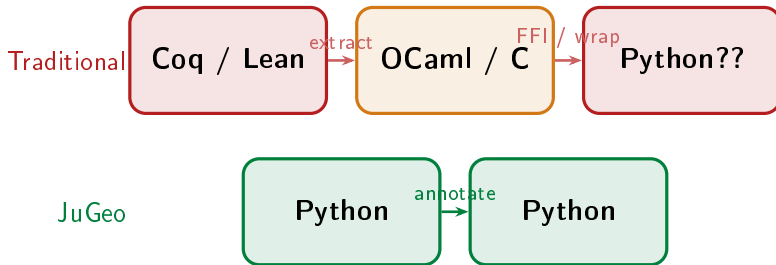
JuGeo Tactics

- Operate on **proof geometry**
- Coordinates, covers, trust, obstructions
- **Adaptive control** selects tactics



Tactics are **semantic moves** on a geometric proof state, not syntactic rewrites.

Bet 4 — Python-Native Over Extraction



- If you **ship Python**, the proof should **be Python**
- Annotated with coordinates and support
- Any verifier can reconstruct the argument from the source

Comparison with Existing Systems (1/3)

Feature	Lean 4	Coq	F*	JuGeo
Dependent types	✓	✓	✓	—
Refinement types	—	—	✓	✓
Effect system	—	—	✓	✓
SMT integration	limited	—	✓	✓
Tactic framework	✓	✓	limited	✓

*JuGeo replaces dependent types with **geometric coordinates + trust***

Comparison with Existing Systems (2/3)

Feature	Lean 4	Coq	F*	JuGeo
Code target	C / IR	OCaml	C / F#	Python
Trust model	binary	binary	binary	7 levels
Evidence channels	1	1	2	4+
Failure model	error msg	error msg	error msg	obstruction
Repair guidance	—	—	—	✓
Project-wide	limited	limited	limited	✓

Failures as **obstructions** enable **repair frontiers** — not just error text.

Comparison with Existing Systems (3/3)

Feature	Lean 4	Coq	F*	JuGeo
Exceptions	—	—	partial	✓
Async / await	—	—	—	✓
Generators	—	—	—	✓
Metaclasses	—	—	—	✓
Context managers	—	—	—	✓
AI integration	—	—	—	✓
Backpressure	—	—	—	✓

Python's dynamic features are **first-class citizens**, not afterthoughts.

What This Means for Python Developers

No new language

Write Python
verify Python

No extraction

Source = proof and

Mixed evidence

Tests + proofs
+ AI together

For the first time

Formal verification that works **with** Python, not against it.

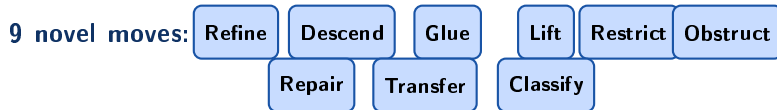
Exceptions, generators, async, metaclasses — all supported.

Your existing test suite counts as evidence from day one.

What This Means for Verification Researchers

New Foundations

- **Sheaf-theoretic proofs** — sections, descent, cohomology
- **Multi-channel evidence** — trust algebra over 7 levels
- **Structured failures** — obstructions classified by cohomology



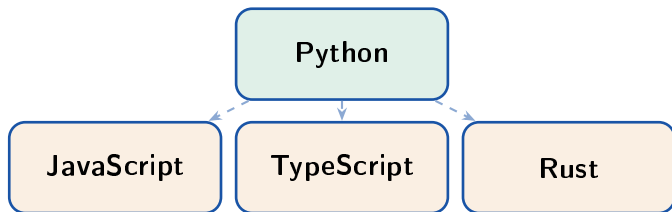
Proof failures become **mathematical objects** you can analyze, not just error messages.

What This Means for AI and Code



- AI (Copilot) is a **first-class evidence channel**
- Trust ceiling: AI evidence cannot exceed level 4 without corroboration
- Every AI-sourced claim is **tracked, audited, bounded**

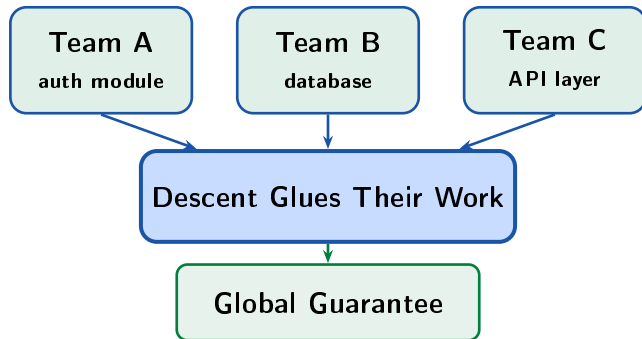
Future: Beyond Python



The geometric framework
is **language-agnostic**.
Only the runtime model changes.

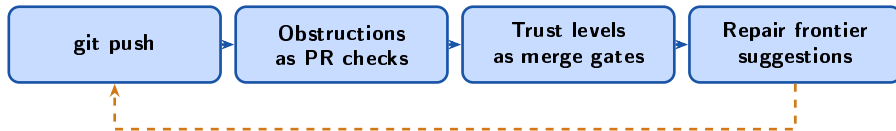
- Coordinates, covers, trust, obstructions — all transfer
- Each language needs its own **runtime adapter**
- Community contributions welcome

Future: Distributed Verification



- Multiple teams verify different parts of the codebase
- **Trust tracks** who verified what, when, and how
- Descent ensures local proofs compose into global correctness

Future: JuGeo in CI/CD



CI/CD Concept	JuGeo Mapping
PR check	Obstruction report
Merge gate	Minimum trust level
Auto-suggestion	Repair frontier
Dashboard	Trust heatmap over coordinates

Getting Started

Install and run in 30 seconds

```
pip install -e .  
pytest test_examples/ -v  
python3 test_algorithms.py
```

Requirements

- Python ≥ 3.10
- No external solvers needed for basic usage
- Optional: Z3 for SMT channel, Copilot for AI channel

From zero to verified in one `pip install`

Five Things to Remember

1

Proofs are **geometric**, not terms

2

Trust has **7 levels**, not 2

3

Failures are **objects**, not errors

4

Python in, Python out

5

AI is a **bounded evidence channel**

*“Proofs are not terms.
Proofs are geometric:
they are sections over sites,
glued by descent,
and observable through their support.”*

Thank you!

JuGeo — Judgment Geometry for Python

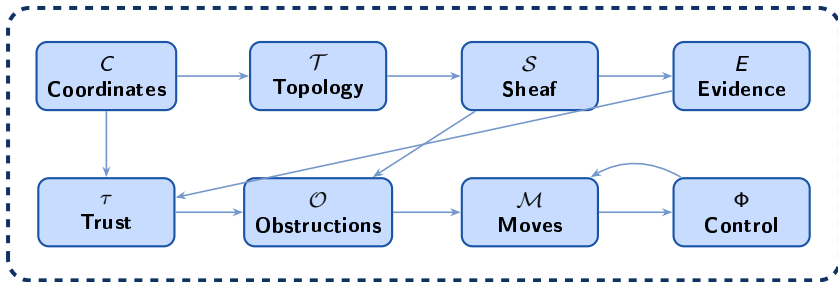
`github.com/jugeo/jugeo`

`contact@jugeo.dev`

Documentation: `jugeo.readthedocs.io`

Questions?

The Judgment 8-Tuple



?

Geometry in, geometry out — all the way down.